TAIL-SENSITIVE LEAKAGE ASSESSMENT FOR ML-KEM: COMBINING EXTREME VALUE THEORY WITH DEEP LEARNING-BASED PROFILED ATTACKS

MACSE698 – Dissertation-1

Submitted in partial fulfillment of the requirements for the degree of

Master of Technology in Cyber Security

in

Department of Cyber Security and Intelligent Systems

by

G MADHUSUDAN

25MCY0001

Under the guidance of

Dr. Jeyanthi N

School of Computer Science Engineering and Information Systems

VIT, Vellore



November, 2026

TAIL-SENSITIVE LEAKAGE ASSESSMENT FOR ML-KEM: COMBINING EXTREME VALUE THEORY WITH DEEP LEARNING-BASED PROFILED ATTACKS

MACSE698 – Dissertation-1

Submitted in partial fulfillment of the requirements for the degree of

Master of Technology in Cyber Security

in

Department of Cyber Security and Intelligent Systems

by

G MADHUSUDAN

25MCY0001

Under the guidance of

Dr. Jeyanthi N

School of Computer Science Engineering and Information Systems

VIT, Vellore



November, 2026

DECLARATION

I hereby declare that the MACSE698 – Dissertation-1 entitled "**Tail-Sensitive Leakage Assessment for ML-KEM: Combining Extreme Value Theory with Deep Learning-Based Profiled Attacks**" submitted by me, for the award of the degree of Master of Technology in Cyber Security in Department of Cyber Security and Intelligent Systems, School of Computer Science Engineering and Information Systems to VIT is a record of bonafide work carried out by me under the supervision of Dr. Jeyanthi N, Professor Grade 1, SCORE, VIT, Vellore.

I further declare that the work reported in this dissertation has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidate

CERTIFICATE

This is to certify that the MACSE698 – Dissertation-1 entitled "**Tail-Sensitive Leakage Assessment for ML-KEM: Combining Extreme Value Theory with Deep Learning-Based Profiled Attacks**" submitted by G Madhusudan (25MCY0001), SCORE, VIT, for the award of the degree of Master of Technology in Cyber Security in Department of Cyber Security and Intelligent Systems, is a record of bonafide work carried out by him under my supervision during the period 06.07.2026 to 03.11.2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The dissertation fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Date:

Signature of the VIT-SCORE Guide

Internal Examiner **External Examiner**

Head of the Department

Department of Cyber Security and Intelligent Systems

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to my Dissertation-1 guide **Dr. Jeyanthi N, Professor Grade 1**, School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore for her constant guidance, continual encouragement, in my endeavor. My association with her is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and an expert in the field of cryptography and information security.

I would like to express my heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Dr. Sankar Viswanathan, Dr. Sekar Viswanathan**, Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**; Pro-Vice Chancellor **Dr. Partha Sharathi Mallick**; and Registrar **Dr. Jayabarathi T**.

My whole-hearted thanks to Dean **Dr. Daphne Lopez**, School of Computer Science Engineering and Information Systems, Head, Department of Cyber Security and Intelligent Systems **Dr. Selva Rani B**, MTech-Cyber Security Project Coordinator **Dr. Lawanya Shri M**, SCORE School Project Coordinator **Dr. Karthikeyan P**, all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways.

I also acknowledge the Ledger Donjon team for publicly releasing the ML-KEM electromagnetic side-channel dataset on which this work is based.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my dissertation successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the dissertation.

Place: Vellore

Date: **G Madhusudan**

EXECUTIVE SUMMARY

ML-KEM, the post-quantum key-encapsulation standard (FIPS 203), can leak its secrets through electromagnetic emanations when it runs on real hardware. This dissertation measures that leakage on the public Ledger Donjon dataset, which holds pqm4, reference and first-order masked implementations (800,000 traces), using conventional leakage statistics, extreme value theory (EVT) and deep-learning profiled attacks. The t-test and signal-to-noise analysis show strong leakage in both unprotected implementations, while the masked secret shows no first-order leakage, a result backed by a share-level positive control. A Generalised Pareto Distribution cut-off removes 46–70% of the attack points with little loss of accuracy. A rebuilt deep-learning pipeline (training-only point selection, windowed inputs, three seeds, calibration) lifts validation accuracy from 50.5% to 94.1% for the reference code and from 31.4% to 39.6% for pqm4 with 50 points, matching what 100 or 150 points achieve. On the full reference attack set the accuracy stays at 94.65%, and a single trace reveals the secret's Hamming-weight class 89.2% of the time. On pqm4, every model picks the wrong class, and the cause turns out to be a shift between the profiling and attack recording sessions, not the model or the secret value.

CONTENTS

			Page No.
	Acknowledgement		v
	Executive Summary		vi
	Table of Contents		vii
	List of Figures		ix
	List of Tables		x
	Abbreviations		xii
	Symbols and Notations		xiii
1	**INTRODUCTION**		1
	1.1	Objective	1
	1.2	Motivation	2
	1.3	Background	3
	1.4	Organisation of the Report	9
2	**DISSERTATION DESCRIPTION AND GOALS**		10
	2.1	Problem Description	10
	2.2	Literature Survey	10
	2.3	Findings from the Literature Survey	13
	2.4	Base Paper	14
	2.5	Research Gaps	15
	2.6	Goals and Research Questions	16
	2.7	Scope	16
3	**TECHNICAL SPECIFICATION**		18
	3.1	Hardware and Compute Environment	18
	3.2	Software Stack	18
	3.3	Dataset	19
	3.4	Class Distribution of the Profiling Sets	20
	3.5	Functional Requirements	21
	3.6	Non-Functional Requirements	22
4	**DESIGN APPROACH AND DETAILS**		23

	4.1	Design Approach / Materials and Methods	23
	4.2	Codes and Standards	27
	4.3	Constraints, Alternatives and Tradeoffs	28
5	**SCHEDULE, TASKS AND MILESTONES**		30
	5.1	Stages and Tasks	30
	5.2	Milestones	31
	5.3	Changes to the Plan	31
6	**DISSERTATION DEMONSTRATION**		33
	6.1	Stages 1–10: Leakage Assessment, EVT Cut-Off and First Attack Evaluation	33
	6.2	Stage 11: Maximum-Accuracy Profiled Attack	39
7	**RESULTS AND DISCUSSION**		46
	7.1	Validation Accuracy	46
	7.2	Test-Split Results	47
	7.3	Attack-Set Results	49
	7.4	Findings on the EVT Cut-Off	54
	7.5	Comparison With the Base Paper and Related Work	54
	7.6	Testing and Validation	55
	7.7	Limitations	56
	7.8	Contributions	57
8	**SUMMARY**		59
	8.1	Objectives and Outcomes	59
	8.2	Answers to the Research Questions	60
	8.3	Recommendations	60
	8.4	Future Work	61
9	**REFERENCES**		62
	APPENDIX A		

LIST OF FIGURES

Figure No.	Title	Page No.
1.1	Profiled Side-Channel Attack Workflow	5
1.2	Hamming-Weight Class Prior Over Coefficients Modulo 3329	6
3.1	Hamming-Weight Class Shares in the Profiling Sets Compared With the Theoretical Prior	21
4.1	System Architecture of the Complete Pipeline	23
4.2	Stage 11 Data Flow From Raw Traces to Attack Decision	24
4.3	Use Case Diagram	27
4.4	Component (Class) Diagram	27
5.1	Project Roadmap by Stage and Review	32
6.1	Fitted GPD Survival Curves of the |t| Tail, pqm4 and ref	36
6.2	Training-Split SNR and Selected Points, pqm4	42
6.3	Training-Split SNR and Selected Points, ref	42
7.1	Validation Accuracy Versus Number of Points	46
7.2	Confusion Matrix, ref CNN K = 50 (Three-Seed Ensemble), Test Split	48
7.3	Confusion Matrix, pqm4 MLP K = 50 (Three-Seed Ensemble), Test Split	48
7.4	Guessing Entropy on the Full Attack Sets	50
7.5	Expected Hamming Weight by Trace Group, pqm4 and ref	53
7.6	Bit Weights of the Learned Leakage Model: pqm4 Bits and the ref Range	53

LIST OF TABLES

Table No.	Title	Page No.
1.1	ML-KEM Parameter Sets	3
2.1	Literature Survey	11
2.2	Base Paper Compared With This Work	15
2.3	Research Gaps and How This Work Addresses Them	15
3.1	Hardware and Compute Environment	18
3.2	Software Stack	18
3.3	Dataset Summary	19
3.4	Hamming-Weight Class Counts in the Profiling Sets	20
4.1	Pipeline Modules	25
4.2	Codes and Standards	27
4.3	Design Alternatives and Tradeoffs	29
5.1	Stages, Tasks and Milestones	30
6.1	Baseline Leakage Results	34
6.2	GPD Fits at Tail Fraction 0.02	35
6.3	Tail-Fraction Sensitivity	35
6.4	Worked Example of the EVT Score for the Strongest Sample	36
6.5	Conventional Versus EVT-Based Point Selection	36
6.6	Stage 8 CNN Validation Accuracy	37
6.7	Stage 9 Attack Evaluation (First 10,000 Traces)	38
6.8	Stage 11.1 Stratified Split	39
6.9	Stage 11.2 Point Selection Summary	40
6.10	Expected SNR of a Non-Leaking Sample (Noise Floor), Training Split	41
6.11	Stage 11.4 Profiling-Versus-Attack Drift Check	42
6.12	Stage 11 Model Architectures	43
6.13	Stage 11 Training Settings	44
7.1	Validation Accuracy Over Three Seeds	46

Table No.	Title	Page No.
7.2	Full Attack-Set Results, New Models	49
7.3	Old (Stage 8) Versus New (Stage 11) Models on the Full Attack Set	51
7.4	pqm4 Diagnosis: Expected Hamming Weight by Session	52
7.5	Comparison With the Base Paper and Related Work	54
7.6	Testing and Validation Checks	55
8.1	Objectives and Their Outcome	59

ABBREVIATIONS

Abbreviation	Expansion
AES	Advanced Encryption Standard
CMOS	Complementary Metal-Oxide-Semiconductor
CNN	Convolutional Neural Network
CPA	Correlation Power Analysis
CPU	Central Processing Unit
DL	Deep Learning
EM	Electromagnetic
EVT	Extreme Value Theory
FIPS	Federal Information Processing Standard
FO	Fujisaki–Okamoto (transform)
FPGA	Field-Programmable Gate Array
GE	Guessing Entropy
GPD	Generalised Pareto Distribution
GPU	Graphics Processing Unit
HW	Hamming Weight
KEM	Key-Encapsulation Mechanism
LWE	Learning With Errors
ML-KEM	Module-Lattice-Based Key-Encapsulation Mechanism
MLP	Multilayer Perceptron
NIST	National Institute of Standards and Technology
NTT	Number-Theoretic Transform
PAC	Probably Approximately Correct
PI	Perceived Information
POI	Point of Interest
PQC	Post-Quantum Cryptography
RNN	Recurrent Neural Network
SCA	Side-Channel Analysis
SNR	Signal-to-Noise Ratio

Abbreviation	Expansion
SR	Success Rate
TVLA	Test Vector Leakage Assessment
UML	Unified Modeling Language

SYMBOLS AND NOTATIONS

Symbol	Meaning
q	ML-KEM modulus, 3329
n	Polynomial degree, 256
k	Module rank (2, 3 or 4)
η_1, η_2	Noise (distribution) parameters of an ML-KEM parameter set
d_u, d_v	Ciphertext compression parameters of an ML-KEM parameter set
a, b	Operands of the NTT-domain base multiplication
$HW(x)$	Hamming weight (number of set bits) of x
c	Hamming-weight class, 0 to 11
$\pi(c)$	Prior probability of class c in the training data
t	Welch's t-statistic at a sample point
μ_1, μ_2; s_1^2, s_2^2; N_1, N_2	Means, variances and trace counts of the two groups in the t-test (Eqn. 1.1)
μ_c, μ, σ_c^2	Mean of class c, overall mean and variance of class c (Eqn. 1.2)
w_c	Weight of class c in the weighted SNR (Eqn. 1.2)
u	GPD threshold
ξ	GPD shape parameter
σ	GPD scale parameter
X, Y, y	Leakage statistic, its excess over u, and an excess value (Eqn. 1.3)
K	Number of selected points (windows)
N	Number of traces
N_c	Number of training traces in class c (Eqn. 6.2)
C	Number of Hamming-weight classes, 12 (Eqns. 6.1–6.2)
T	Calibration temperature
$q(c \mid x)$	Model probability of class c given trace x
x_i, y_i	The i-th trace and its true class
$S_n(c)$	Summed log-probability of class c over n attack traces (Eqn. 1.4)

Symbol	Meaning
$H(\pi)$	Entropy of the class prior, used in perceived information (Eqn. 1.5)
$e_i(c)$	Prior-corrected evidence of trace i for class c: $\log q(c \mid x_i) - \log \pi(c)$ (Eqn. 1.6)

CHAPTER 1

INTRODUCTION

Cryptography is in the middle of its move to post-quantum algorithms (PQC). In August 2024 the National Institute of Standards and Technology (NIST) released FIPS 203, the Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM), which grew out of CRYSTALS-Kyber (National Institute of Standards and Technology, 2024). Its security rests on the Module Learning With Errors (Module-LWE) problem, for which no efficient classical or quantum algorithm is known, and it already appears in hybrid key exchanges in web browsers and cryptographic libraries.

A scheme that is secure on paper is not automatically secure in silicon. A device running ML-KEM uses power. Gives off electromagnetic (EM) fields that change based on the data it's processing. An attacker who records these signals might be able to figure out values. Without even looking at the math behind the encryption. This is called side-channel analysis or SCA.

This dissertation studies the EM leakage from three ML-KEM implementations found in the Ledger Donjon dataset (Magazin & Abdellatif 2026). It combines statistical methods for checking leakage deep-learning-based profiled attacks and extreme value theory (EVT). EVT is a part of statistics that focuses on extreme events, in a distribution. Not the average behavior.

The idea is straightforward: security often depends on events. Like one leaky sample point, one lucky attack or one misleading signal.. Most tools used to test side-channel security are built around averages. This work looks at what happens when we pay attention to the extremes instead.

1.1 Objective

The main objective is to assess the side-channel leakage of ML-KEM implementations using both conventional statistics and EVT, and to check that assessment with deep-learning profiled attacks evaluated on independent attack recordings. The specific objectives are:

1. To carry out Hamming-weight-based leakage assessment (Welch's t-test and SNR) on the complete profiling sets of two unprotected implementations (pqm4 and reference) and one first-order masked implementation.

2. To fit a Generalised Pareto Distribution (GPD) tail model to the leakage statistic and use it as a data-driven cut-off for points of interest, comparing it with a fixed top-50 selection.

3. To build convolutional neural network (CNN) profiled attacks on the selected points and test them on fixed-key attack traces.

4. To check the ground truth and the pairing of traces with labels at every stage, and to fix any errors found.

5. To create the precise attack profile possible under a protocol that has no leaks. This means choosing training points keeping the test set completely separate using many different random seeds and making sure probabilities are properly adjusted. The goal is to discover how few points are needed for this.

6. To evaluate that attack on the complete fixed-key attack sets and explain any failure.

1.2 Motivation

Three observations led to this work. First ML-KEM is being rolled out now so ML-KEM implementations need testing against attacks, with tools that are both sensitive and statistically sound. Welch's t-test, known here as Test Vector Leakage Assessment (TVLA), is the usual first check, but its fixed threshold of $|t| > 4.5$ is applied separately at thousands of correlated sample points. On this very dataset, the rule raises false alarms far more often than its nominal level suggests when results are checked repeatedly (Feretzakis & Papaspyridis, 2026), and the distribution of the related SNR statistic was worked out only recently (des Noes, 2024).

Second, the number of points of interest handed to an attack is normally set by an arbitrary rule such as “the top 50 points”, although too many points add noise and cost and too few throw information away. The base paper compared deep-learning

models with and without t-test-based point filtering on a Kyber accelerator and found that filtering sped up training without hurting accuracy, but it used a fixed selection rule and only an unprotected target (Chinbat et al., 2025). A data-driven rule for how many points really carry leakage is missing.

Third, attack results are often reported with metrics that can mislead. Deep-learning scoring metrics break down under non-uniform class priors (Rousselot et al., 2026), neural networks are over-confident unless calibrated (Nouraniboosjin & Ganji, 2024), and bounds on how many traces an attacker needs rest on strong assumptions (Béguinot et al., 2025; Nouraniboosjin & Ganji, 2026). A model that looks good on validation data can also fail on the victim's recordings. EVT has been used for software timing side channels (Baez et al., 2025), but none of the works reviewed uses it to decide how many leaking points an EM attack should keep.

1.3 Background

1.3.1 ML-KEM

ML-KEM is a key-encapsulation mechanism (KEM). The receiver publishes an encapsulation key, the sender uses it to produce a ciphertext and a shared key, and the receiver gets the same shared key back from the ciphertext using its secret decapsulation key. Below is a Module-LWE public-key encryption scheme. This scheme becomes safe against chosen-ciphertext attacks thanks, to the Fujisaki-Okamoto (FO) transform. When the receiver decapsulates the receiver first decrypts the ciphertext. Then the receiver re-encrypts the result. Then the receiver compares the two ciphertexts. This re-encryption check makes decapsulation a data-dependent computation that can leak information. All arithmetic is done on polynomials that have 256 coefficients and are taken modulo the prime 3329. These polynomials are put together into vectors and matrices that have dimension k. The module rank determines the security level. Table 1.1 lists the three parameter sets.

Table 1.1 ML-KEM Parameter Sets

Parameter	ML-KEM-512	ML-KEM-768	ML-KEM-1024
k	2	3	4
η1	3	2	2
η2	2	2	2
du	10	10	11
dv	4	4	5
Encapsulation key (bytes)	800	1,184	1,568
Decapsulation key (bytes)	1,632	2,400	3,168
Ciphertext (bytes)	768	1,088	1,568
NIST security category	1	3	5

The masked implementation in the dataset stores a 2,400-byte unmasked key and 1,088-byte ciphertexts, which matches ML-KEM-768. Since every parameter set uses q = 3329 and n = 256, the single-coefficient analysis in this report does not depend on k.

1.3.2 The Number-Theoretic Transform and Base Multiplication

Multiplying two polynomials of degree 256 directly would need around 65,000 coefficient multiplications. That's why ML-KEM uses the NTT domain to make things faster. With q = 3329 and n = 256 only a partial NTT is possible. So the multiplication breaks down into 128 multiplications, each of degree one. Each of these multiplications takes two coefficients. One from each input polynomial. And combines them modulo q. During decapsulation the result of each of these depends directly on the coefficients.

The dataset records each base multiplication, written as r = a × b. In the pqm4 and reference implementations operand a is the secret. It stays the same across all the fixed-key cases while b changes. In the version the secret is, in operand b, which is split into two arithmetic shares: b = b0 + b1 mod q. This report focuses on coefficient 0 of the operand.

1.3.3 Side-Channel Attacks

A side-channel attack uses clues that leak out while a device is working. In CMOS logic each bit shift pulls some current. The power usage and the electromagnetic field change depending on the data that is being processed. The dataset was recorded with a near-field EM probe on an STM32F407 ARM Cortex-M4 microcontroller (Magazin & Abdellatif, 2026). Each recording, called a trace, is a time series of EM amplitude samples covering one run of the target operation.

Non-profiled attacks, such as correlation power analysis (CPA), test key guesses directly against traces from the target device using a leakage model. Profiled attacks work in two steps: the attacker first learns the leakage on a device they control, using traces with known secrets (the profiling set), and then applies what was learned to traces from the target device, whose secret is unknown (the attack set). Profiled attacks are the strongest attack model and serve as the reference for worst-case evaluation, and deep neural networks have become the main profiling method (Picek et al., 2023). Fig. 1.1 shows the workflow followed in this dissertation.

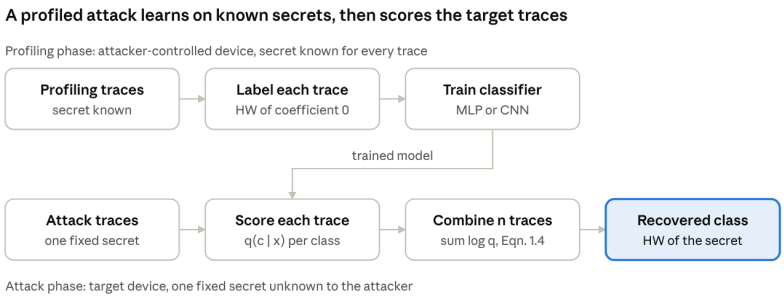


Fig. 1.1 Profiled Side-Channel Attack Workflow

1.3.4 Leakage Models and the Hamming Weight

A leakage model says which function of the secret the device leaks. For microcontrollers the most common choice is the Hamming weight (HW), the number of 1 bits in a value. Coefficients modulo 3329 are 12-bit values between 0 and 3328, so HW(x) runs from 0 to 11, which gives 12 classes.

The HW fits the physics, because values with the same number of set bits draw similar current, and it gives 12 classes with plenty of traces in each. One HW observation narrows a coefficient down to a group of values (HW 6 alone covers 784

of the 3,329), so the attacks here recover the HW class of the secret coefficient rather than its exact value.

The HW classes are not equally likely. If coefficients are spread uniformly modulo q, the prior of a class is the number of values with that HW divided by 3,329. Those counts are 1, 12, 66, 219, 478, 728, 784, 596, 313, 108, 22 and 2 for HW 0 to 11, and Fig. 1.2 plots the resulting probabilities. The most common class, HW 6, has probability 0.2355, so always answering HW 6 is already right 23.6% of the time; that, and not 1/12 = 8.3%, is the baseline to beat. The entropy of the prior, 2.756 bits, is the most a single trace could ever tell us about the HW class.

The prior peaks at HW 6 (23.55%) and HW 5 (21.87%), while the four outer classes HW 0, 1, 10 and 11 together account for only 1.1% of all values. These classes have the fewest traces in every split and are the hardest to learn and to evaluate, which comes up again in the per-class results of Chapter 7.

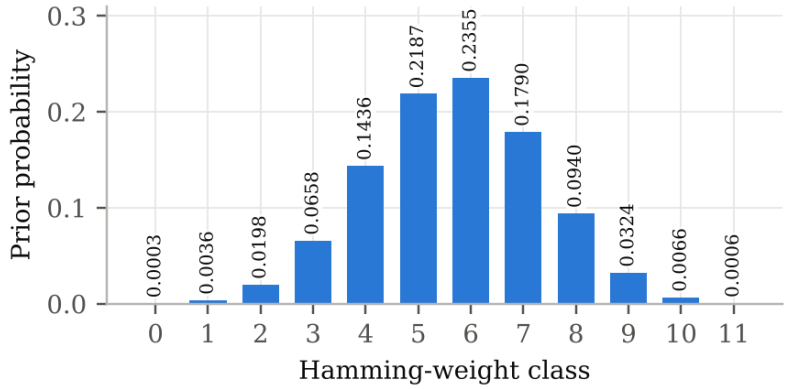


Fig. 1.2 Hamming-Weight Class Prior Over Coefficients Modulo 3329

1.3.5 Conventional Leakage Assessment

Leakage assessment asks whether a trace depends on the secret, and if so, where. This report uses two statistics throughout.

Welch's t-test compares the mean trace of two groups at each sample point. For group means $\mu 1$ and $\mu 2$, variances $s1^2$ and $s2^2$ and sizes N1 and N2, the statistic is given by Eqn. 1.1.

$$t = \frac{\mu_1 - \mu_2}{\sqrt{s_1^2/N_1 + s_2^2/N_2}} \quad \text{(Eqn. 1.1)}$$

In TVLA practice, a value of $|t| > 4.5$ at any sample is taken as a sign of leakage. Here the two groups are traces whose secret HW's at or below the median (HW $\leq$ 6) and traces whose secret HW is above that median. This makes it a secret-dependent (t-test rather, than a fixed-versus-random test.

The signal-to-noise ratio (SNR) sets how far the mean trace moves between the 12 HW classes against how much traces vary inside a class. With class weights w_c, class means μ_c, overall mean μ and class variances σ_c², the weighted SNR used in Stage 11 is given by Eqn. 1.2.

$$\mathrm{SNR} = \frac{\sum_c w_c(\mu_c - \mu)^2}{\sum_c w_c \sigma_c^2} \quad \text{(Eqn. 1.2)}$$

Even when nothing leaks, Eqn. 1.2 averages about (C − 1)/N for C classes and N traces, simply because sample means are noisy. Chapter 6 uses this noise floor, roughly 7×10^{-5} for pqm4 and 1.4×10^{-4} for ref, to tell real leakage apart from noise.

1.3.6 Extreme Value Theory and the Generalised Pareto Distribution

EVT describes how a distribution behaves beyond a high threshold. Its key result is that, for a broad family of distributions, the excess Y = X − u of a variable X over a high enough threshold u follows, approximately, a Generalised Pareto Distribution with the survival function of Eqn. 1.3.

$$P(Y > y \mid X > u) = \left(1 + \frac{\xi y}{\sigma}\right)^{-1/\xi}, \quad y > 0 \quad \text{(Eqn. 1.3)}$$

The scale $\sigma > 0$ controls the spread of the tail and the shape ξ its type: $\xi > 0$ is a heavy (power-law) tail, $\xi = 0$ the exponential limit and $\xi < 0$ a bounded tail with a finite upper end. Combined with the observed probability of exceeding u, Eqn. 1.3 gives the probability of values well beyond the data, extending a tail from its observed shape instead of assuming a normal distribution.

Every GPD analysis has to pick the threshold u, balancing bias (u too low) against variance (u too high); Murphy, Tawn and Varty (2023) develop an automated way to choose it with uncertainty estimates. The analysis also has to check how stable the fit is, which this report does by looking at how the fitted shape changes with the tail fraction (Section 6.1.4).

1.3.7 Deep-Learning Profiled Attacks

In a deep-learning profiled attack, a neural network is trained on profiling traces x with known labels y (here the HW class) to output a probability q(c | x) for each class c, by minimising the cross-entropy loss, the average of −log q(y | x). Two architectures are used: a multilayer perceptron (MLP), which connects every input sample to every neuron, and a convolutional neural network (CNN), which slides small filters along the time axis to spot leakage shapes even when they shift slightly.

The attack then combines n attack traces x1, …, xn that share the same unknown secret. Each class c gets a score equal to the sum of log-probabilities over the traces (Eqn. 1.4), and the class with the highest score is the guess.

$$S_n(c) = \sum_{i=1}^{n} \log q(c \mid x_i) \qquad \text{(Eqn. 1.4)}$$

1.3.8 Evaluation Metrics

Accuracy is how many traces get the class the most of all the possible classes. It shows how well a single trace is doing. This number only makes sense when we compare it to the majority-class baseline, which's 23.6%. Guessing entropy (GE) is the average rank of the true class after n traces, taken over random orderings of the attack traces; GE = 0 means the true class comes first. When I look at the numbers Success rate (SR) tells me how often the right class is placed first in a n-trace attack. Perceived information (PI) expressed in bits per trace shows how many bits per trace the model’s probability predictions cut down in my uncertainty, about the class as laid out in Eqn. 1.5 And using the entropy H(π).

$$\mathrm{PI} = H(\pi) + \frac{1}{N}\sum_{i=1}^{N} \log_2 q(y_i \mid x_i)$$

(Eqn. 1.5)

If the class prior is not uniform the sum in Eqn. 1.4 Adds n · log π(c) for each class. This pulls the decision toward the classes. Rousselot and colleagues in 2026 show that standard scoring metrics do not work when class priors are uneven. Prior correction eliminates this term by scoring each class with the evidence, from Eqn. 1.6.

$$e_i(c) = \log q(c \mid x_i) - \log \pi(c)$$

(Eqn. 1.6)

I have observed that neural networks are often over-confident meaning that networks predicted probabilities are higher, than the real accuracy. Temperature scaling divides the output logits by a number T. This number T is fitted on validation data by minimising the log-likelihood. Temperature scaling has been shown to make side-channel attacks more efficient (Nouraniboosjin & Ganji 2024). It matters in this work because perceived information, prior-corrected scoring and success rate all use the probabilities themselves, not just the predicted class.

1.3.9 Masking

Masking is the way to protect against side-channel attacks. The secret is broken into d + 1 pieces. These pieces, when added together (using modulo q) equal the secret. Each piece is processed separately during computation.

In a first-order system, which uses two shares no single piece gives away any information about the secret. So even if each piece leaks a lot of data an attacker shouldn't be able to figure out the secret from one. They'd need to combine both shares to get anywhere.

That's where second-order attacks come in. These attacks look for patterns when two shares are combined. Now deep learning methods have been used to break Kyber masking up to fifth order. This was shown in a study, by Dubrova et al. In 2023.

1.4 Organisation of the Report

Chapter 2 talks about the problem what's already been written on it the base paper, what's missing and what we want to achieve. Chapter 3 goes into the details. Chapter 4 is about the design. Chapter 5 shows the schedule. Chapter 6 walks through the implementation step, by step. Chapter 7 shares the results. Chapter 8 wraps everything up. Chapter 9 lists the references. Appendix A has sample code, extra tables and a file map.

CHAPTER 2

DISSERTATION DESCRIPTION AND GOALS

2.1 Problem Description

Side-channel evaluation has two parts. Leakage assessment asks whether and where traces depend on the secret, and attack evaluation asks how quickly a trained model recovers it. Both parts rely on decisions made by fixed rules.

In leakage assessment, the t-test and SNR give every sample a score, and the highest-scoring points of interest (POIs) are passed to an attack. How many are kept is decided in advance (the top 50, for example) or by a threshold such as $|t| > 4.5$. That threshold is applied separately at 10,000 to 20,000 correlated samples, so the false-alarm rate for the trace as a whole is not controlled.

In attack evaluation, unequal class priors (Fig. 1.2) let accuracy, guessing entropy and success rate show success for a model that has learned nothing. A model that always predicts HW 6 ranks it first, so if the secret happens to be HW 6, its guessing entropy reaches zero.

Finally, profiling and attack traces are recorded in different sessions. A model that does well on held-out profiling traces can still fail on the attack set if the set-up changed between recordings, and reporting validation results alone hides this.

The problem this work addresses is therefore to build an ML-KEM leakage-assessment and profiled-attack pipeline in which a data-driven tail rule decides how many points of interest to keep, attacks are tested on independent fixed-key recordings with prior-aware metrics, and failures are explained rather than hidden.

2.2 Literature Survey

Eighteen papers from 2023 onwards were reviewed, covering side-channel attacks on Kyber and ML-KEM, deep-learning profiled attacks and how they are evaluated, leakage assessment and EVT. Five come from the Review 2 survey and the others are recent work on leakage detection, attack metrics and EVT. For each paper, Table 2.1 lists the method and findings (merits) and the gaps relative to this project (demerits).

Table 2.1 Literature Survey

No.	Author(s), Year, Title and Source	Merits (Methodology and Findings)	Demerits (Gaps)
1	Chinbat et al. (2025). Comparative deep learning-based side-channel analysis of an FPGA-based CRYSTALS-Kyber NTT accelerator. Cryptography (MDPI), 9(4), Art. 64.	Compared 12 deep-learning models (MLP, CNN, RNN) with and without TVLA-based POI filtering on an FPGA Kyber NTT accelerator, 200,000 power traces. Up to 96.64% accuracy; filtering sped training up to 10.5× with no accuracy loss.	Fixed TVLA selection rule; unprotected target only; validation accuracy without a separate fixed-key attack set.
2	Kennaway et al. (2025). An enhanced two-step CPA side-channel analysis attack on ML-KEM. SECRYPT 2025.	Two-step CPA on ML-KEM-512 base multiplication (ChipWhisperer CW308/ STM32F3). Full private-key recovery with 179 traces, no chosen ciphertexts.	Classical CPA only; no deep-learning or leakage-detection layer; no masking; no rule for which points to attack.
3	Dubrova et al. (2023). Breaking a fifth-order masked implementation of CRYSTALS-Kyber by copy-paste. ACM APKC 2023.	Deep-learning attack on Kyber768 masked up to fifth order (ARM Cortex-M4) by recursive learning, copying weights from lower-order networks; messages recovered at every order.	Needs a controlled profiling device and tens of thousands of traces; targets message encoding, not multiplication; no rule for ranking or limiting points.
4	Picek et al. (2023). SoK: Deep learning-based physical side-channel analysis. ACM Computing Surveys, 55(11).	Systematises deep-learning SCA into phases, maps existing work to phases and threat models, and lists open problems.	Survey only; examples mostly AES rather than lattice-based schemes.
5	Ravi et al. (2023). Machine learning based blind side-channel attacks on PQC-based KEMs: A case study of Kyber KEM. ICCAD 2023.	Blind (no ciphertext knowledge) attack on pointwise multiplication with an HW classifier and maximum likelihood (ARM Cortex-M4). Key in 820 traces with a perfect HW classifier, about 7,805 at 95% accuracy.	Shows how classifier accuracy drives attack cost, but no leakage-scoring layer, TVLA/SNR baseline or masked target.

No.	Author(s), Year, Title and Source	Merits (Methodology and Findings)	Demerits (Gaps)
6	Ganesh et al. (2024). Higher order leakage assessment and neural network-based attack on CRYSTALS-Kyber. SECRYPT 2024.	Univariate central-moment second-order t-test on first-order masked Kyber message decoding (1,000 traces), then a neural-network attack.	No numeric |t| values; no multiplicity treatment; no tail modelling.
7	Magazin & Abdellatif (2026). Open EM side-channel dataset for ML-KEM (Kyber) implementations. IACR ePrint 2026/1851.	Public near-field EM dataset of base multiplication on STM32F407 for reference, pqm4 and first-order masked (mkm4) code, with profiling and fixed-key sets. Textbook CPA on pqm4 needs about 40 traces; bivariate second-order CPA on mkm4 reaches rank 0 after about 200.	No HW labels or train/test split; summary-table trace lengths differ from the files (Section 3.3).
8	Feretzakis & Papaspyridis (2026). The price of peeking: Anytime-valid leakage detection on ML-KEM EM traces. arXiv 2609.33597.	Same Ledger recordings. With random-label designed nulls, repeated checking of |t| > 4.5 raised false alarms in 2.7–12.9% of replicates; an e-process with e-Bonferroni raised none. Detection within 62–316 evaluation traces.	Binary median-split labels; unmasked targets only; union bound ignores sample correlation; no tail model or POI ranking.
9	des Noes (2024). Distribution of signal to noise ratio and application to leakage detection. IACR TCHES 2024(2), 384–402.	Derives the distribution of the SNR under no leakage and detection thresholds, with a Šidák correction for the whole trace.	Trace-wide correction assumes independent samples, which adjacent EM samples are not.
10	Rousselot et al. (2026). Is it really broken? The failure of DL-SCA scoring metrics under non-uniform priors. IACR ePrint 2026/338.	Shows GE and related metrics give false positives under non-uniform priors; proposes an asymptotically optimal distinguisher that divides out the prior; recommends attack subsets of at most 10% of the pool.	Mean behaviour of scores only; no tail model of per-trace evidence.

No.	Author(s), Year, Title and Source	Merits (Methodology and Findings)	Demerits (Gaps)
11	Nouraniboosjin & Ganji (2024). Too hot to be true: Temperature calibration for higher confidence in NN-assisted side-channel analysis. IACR ePrint 2024/071.	Shows NN-based SCA models are over-confident and that temperature scaling makes the resulting attacks more efficient.	Calibration only; effect on rare, extreme evidence values not studied.
12	Nouraniboosjin & Ganji (2026). How many traces suffice? PAC guarantees for profiled and non-profiled side-channel analysis. IACR ePrint 2026/1770.	Defines a per-trace evidence margin between the true and a competing candidate and bounds the traces needed with Bernstein's inequality.	Uses only the margin's mean, variance and clipping range; tighter guarantees need more distributional assumptions.
13	Béguinot et al. (2025). Scalable information theoretic evaluation of the rank statistics in side-channel attacks. IACR ePrint 2025/1316.	Lower bound on traces from single-trace mutual information; notes a long-tailed key-rank distribution, close to a truncated Zipf law.	Long tail used only to argue tightness; no tail model fitted.
14	Mikulec et al. (2026). Beyond TVLA: Anderson–Darling leakage assessment for neural network side-channel leakage detection. arXiv 2603.18647.	Per-sample two-sample Anderson–Darling test, sensitive beyond the mean including tails; threshold about 11.18 ($\alpha \approx 6.8 \times 10^{-6}$).	Two equal-size groups only; trace-wide maximum against a pointwise threshold, uncorrected over 108,300 samples.
15	Meunier (2026). A unified framework for statistical side-channel distinguishers: From leakage assessment to key recovery. IACR ePrint 2026/2041.	Compares 18 statistics for detection and key recovery; tail-weighted Anderson–Darling stalls near random guessing on masked AES.	AES only; no extreme value modelling; evidence against tail-weighted tests on masked data.

No.	Author(s), Year, Title and Source	Merits (Methodology and Findings)	Demerits (Gaps)
16	Baez et al. (2025). Risk estimation in differential fuzzing via extreme value theory. arXiv 2511.02927.	Peaks-over-threshold GPD models of worst-case cost differences found by fuzzing, including software timing side channels, with bootstrap threshold selection.	Software timing, not power/EM; EVT degrades for discrete metrics and over-estimated in some settings.
17	Peschel et al. (2026). permApprox: A general framework for accurate permutation p-value approximation. arXiv 2602.22975.	Fits a support-constrained GPD to the tail of a permutation distribution to estimate very small p-values from few permutations.	Applied to genomics and microbiome data, not leakage traces.
18	Murphy et al. (2023). Automated threshold selection and associated inference uncertainty for univariate extremes. arXiv 2310.17999.	Automated GPD threshold selection (EQD) with bootstrap propagation of threshold uncertainty into tail estimates.	General method; coverage can be poor when extrapolating Gaussian-like tails far beyond the data.

2.3 Findings from the Literature Survey

Six findings come out of the survey.

First, deep-learning profiled attacks are now the main profiling method (Picek et al., 2023) and reach high accuracy on unprotected Kyber targets (Chinbat et al., 2025; Ravi et al., 2023), with polynomial multiplication a frequent target (Kennaway et al., 2025; Ravi et al., 2023).

Second, POI selection relies on average-case statistics and fixed rules, such as TVLA filtering (Chinbat et al., 2025). None of the attack papers models the tail of the leakage statistic or lets the data decide how many points to keep.

Third, how to calibrate leakage detection is an open question: on this dataset the $|t| > 4.5$ rule misbehaves under repeated checking (Feretzakis & Papaspyridis, 2026), the SNR's distribution has been derived (des Noes, 2024), and tests that look

beyond the mean are being studied (Mikulec et al., 2026; Meunier, 2026). Their trace-wide corrections, however, assume independent samples or ignore correlation.

Fourth, attack metrics are being questioned. Non-uniform priors cause false positives in guessing entropy (Rousselot et al., 2026), uncalibrated networks are over-confident (Nouraniboosjin & Ganji, 2024), and attack-cost bounds rely on moments of the evidence (Nouraniboosjin & Ganji, 2026) or on mutual information (Béguinot et al., 2025). None of them fits a tail model to the evidence.

Fifth, EVT has been used for software timing side channels (Baez et al., 2025) and for permutation p-values outside security (Peschel et al., 2026), and good tools exist for choosing its threshold (Murphy et al., 2023). Still, none of the works reviewed applies it to EM leakage statistics or to attack evidence.

Sixth researchers study implementations less often except, for Dubrova et al. In 2023 and Ganesh et al. In 2024. In this dataset a second-order CPA attack breaks the implementation after about 200 traces as shown by Magazin and Abdellatif in 2026.

2.4 Base Paper

The base paper, Chinbat et al. (2025) Appears in Table 2.2. It matches point filtering with deep-learning attacks on Kybers NTT arithmetic. The authors say that filtering speeds up training while keeping accuracy the same. I think we can check that by using a data-driven filter on attack data that's separate, from the training set.

Table 2.2 Base Paper Compared With This Work

Aspect	Base paper (Chinbat et al., 2025)	This dissertation
Target	FPGA CRYSTALS-Kyber NTT accelerator	ML-KEM base multiplication on STM32F407 (ARM Cortex-M4), three implementations
Side channel	Power traces	Near-field EM traces (public Ledger Donjon dataset)
Traces	200,000	200k + 200k (pqm4), 100k + 100k (ref), 100k + 100k (masked)
Protection	Unprotected only	Two unprotected and one first-order masked
POI selection	TVLA filtering with a fixed rule	Top-50 vs GPD cut-off (Stages 4–6); training-only weighted SNR, K = 50/100/150 (Stage 11)
Models	12 models (MLP, CNN, RNN variants)	CNN (Stages 7–8); MLP and CNN, three seeds, calibrated (Stage 11)
Best result	96.64% accuracy; up to 10.5× faster training	ref CNN 94.77% validation, 94.65% on full attack set; pqm4 39.61% validation
Evaluation	Classification accuracy	Validation, untouched test split, full fixed-key attack sets; GE, SR, prior correction
Tail analysis	None	GPD on |t| for the POI cut-off (Stages 5–6)

2.5 Research Gaps

Table 2.3 Research Gaps and How This Work Addresses Them

No.	Gap	Evidence from survey	How addressed in this dissertation
G1	POI count fixed by an arbitrary rule	Chinbat et al. (2025)	GPD cut-off vs top-50 (Stages 4–8); accuracy vs K = 50/100/150 (Stage 11)
G2	Attack results reported as validation accuracy only	Chinbat et al. (2025)	Untouched test split and complete fixed-key attack sets (Stages 9, 11.6–11.9)
G3	Metrics misleading under non-uniform HW priors	Rousselot et al. (2026)	Majority-class baseline, null model, prior-corrected scoring and success rate (Stages 9, 11)
G4	Over-confident model probabilities	Nouraniboosjin & Ganji (2024)	Temperature scaling fitted on validation data (Stage 11.6)
G5	Profiling-to-attack session shift not diagnosed	Validation-only reporting in surveyed attacks	Drift analysis and expected-HW diagnosis (Stages 11.4, 11.4b, 11.8c)
G6	Masked implementation rarely included in assessment	Survey finding 6	First-order assessment of mkm4 with share-level positive control (Stages 3, 10)
G7	Ground truth and trace-label pairing rarely audited	Dataset ships no labels (Magazin & Abdellatif, 2026)	Integrity audit and masked label fix (Stage 10)

2.6 Goals and Research Questions

The goals come from the gaps we see:

- Build a full reproducible leakage-assessment pipeline for the three implementations. Start from traces and go all the way to labelled statistics.
- Test if cutting off the tail of a GPD on |t| can shrink the POI set without hurting attack performance.
- Build the accurate profiled attack possible under a leakage-free protocol. Find out how points it really needs to work well.

- Test every model on fixed-key attack data using prior-aware metrics. Report any failures as they happen.

These goals lead to four research questions:

- RQ1: Where and how do the three implementations leak the Hamming weight of secret coefficient 0?
- RQ2: Does a GPD tail cut-off on |t| pick points than a fixed top-50 rule and what does that cost in classification accuracy?
- RQ3: How accurate are MLP and CNN attacks with K = 50, 100 and 150 training-selected points, on validation, test and attack data?
- RQ4: When a model that does well on validation data fails on attack data what causes it?

2.7 Scope

- Three implementations from the Ledger Donjon dataset: pqm4 and reference (unprotected) and mkm4 (first-order masked ML-KEM-768).
- Target: base multiplication; label: HW of coefficient 0 (12 classes); the attack recovers an HW class, not a key.
- Complete profiling sets for assessment and complete fixed-key sets for attacks; the masked implementation is analysed at first order only.
- EVT is used for the POI cut-off (Stages 5–6); the extensions C1–C4 (Section 8.4) and second-order attacks are left for future work.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 Hardware and Compute Environment

All experiments ran in Google Colab notebooks, with the dataset and every intermediate result stored on Google Drive (Table 3.1). Leakage assessment (Stages 1–10) mostly needs memory and disk throughput, so it ran on CPU runtimes, while the deep-learning work of Stage 11 ran on an NVIDIA L4 GPU runtime. Ledger Donjon recorded the traces, so no measurement hardware was needed.

Table 3.1 Hardware and Compute Environment

Component	Specification	Used for
Compute platform	Google Colab (hosted Jupyter notebooks)	All stages
GPU	NVIDIA L4 (24 GB)	Stage 11 training and inference
CPU runtime	Colab standard CPU runtime	Streaming statistics (Stages 3–6, 9–10)
Storage	Google Drive, mounted at /content/drive	Dataset (about 13 GB compressed), intermediate .npz/.npy results, trained models
Target device (dataset)	STM32F407 microcontroller, ARM Cortex-M4	Device on which the traces were recorded by the dataset authors
Acquisition (dataset)	Near-field EM probe, 6.25 GS/s sampling (Magazin & Abdellatif, 2026)	Trace capture by the dataset authors

Session memory is much smaller than the pqm4 profiling set (200,000 traces × 20,000 samples × 2 bytes ≈ 8 GB), so whole-dataset statistics are streamed in 10,000-trace chunks (Section 4.1.2). Sessions can also drop, so every stage saves its outputs to Drive and later stages restart from those files.

3.2 Software Stack

Table 3.2 lists the software used. All code is Python, organised as numbered notebook cells (Appendix A.3).

Table 3.2 Software Stack

Software	Role in the project
Python 3.13	Implementation language
NumPy	Memory-mapped loading of .npy traces, streaming sums and sums of squares, Hamming weights, array operations
SciPy (scipy.stats)	Welch's t-test, GPD fitting (genpareto), distribution functions
scikit-learn	Stratified train/validation/test splitting
TensorFlow / Keras	MLP and CNN construction, training, callbacks (EarlyStopping, ReduceLROnPlateau), inference
Matplotlib	SNR plots, accuracy curves, confusion matrices, guessing-entropy curves, tail plots
Google Colab and Google Drive	Execution environment and persistent storage

3.3 Dataset

The public Ledger Donjon ML-KEM side-channel dataset (Magazin & Abdellatif, 2026) records the base multiplication r = a × b in three implementations: pqm4 (from mupq/pqm4), the reference implementation (from pq-crystals/kyber) and mkm4, a first-order masked ML-KEM-768. Each one has a key profiling set. This uses a secret for each trace. There is also a fixed-key attack set. This uses the secret all the time. Table 3.3 shows the data as checked in this project.

Table 3.3 Dataset Summary

Implementation	Protection	Profiling traces	Attack traces	Samples per trace	Secret operand	Fixed secret coefficient 0 (HW)
pqm4	None	200,000	200,000	20,000	a (mult_a)	679 (HW 6)
ref	None	100,000	100,000	13,000	a (mult_a, stored as bytes, read as int16)	1026 (HW 2)
mkm4 (masked)	First-order arithmetic masking	100,000	100,000	10,000 per share window	b, as shares b = b0 + b1 mod q	1681 (HW 5)

Several properties had to be figured out by looking at the data because the documentation didn’t explain them clearly.

- **Trace lengths.** The files hold 20,000 samples per pqm4 trace and 13,000 per ref trace, which differs from the summary table in the dataset paper.
- **Sample format.** Samples are 8-bit values stored as int16, which means they range from 128, to positive 127.
- **Secret operand.** For pqm4 and ref the metadata does not say which operand is secret. A stays constant across the fixed-key set while b changes. So a is the secret. For mkm4 the documentation says a is public and b is secret.
- **ref metadata encoding.** The reference metadata is stored as little-endian bytes (uint8, 512 bytes for 256 coefficients) and must be read using.view(int16); once done 100% of the values fall in [0, q) unlike the 5.5% under big-endian decoding. The reference metadata is stored as little-endian bytes (uint8, 512 bytes for 256 coefficients) and must be read using.view(int16); once done 100% of the values fall in [0, q) unlike the 5.5%, under big-endian decoding.
- **Masked shares.** Each share has a multiplication that has its own trigger and its own window of 10,000 samples. The fixed-key capture stores the key in a separate file.

The dataset does not include hardware labels or a train/test split so both were created for this project. On Drive the traces are stored in numbered chunks of 10,000. The metadata chunk with the index contains the operands for the traces, in the same row order following the layout shown in Table A.5.

3.4 Class Distribution of the Profiling Sets

Table 3.4 shows the HW class counts of the two unprotected profiling sets side by side with the prior. The proportions line up with the prior by no more than 0.1 percentage point, in each HW class count proving that the secrets are evenly

distributed and that the labels are decoded correctly. I was pleased to see that the numbers matched closely. The table also shows the class imbalance every model faces: HW 0 has 68 pqm4 and 24 ref traces, HW 6 over 47,000 and 23,000.

Table 3.4 Hamming-Weight Class Counts in the Profiling Sets

HW class	pqm4 count	pqm4 %	ref count	ref %	Theoretical prior %
0	68	0.03	24	0.02	0.03
1	703	0.35	358	0.36	0.36
2	4,092	2.05	2,001	2.00	1.98
3	13,248	6.62	6,466	6.47	6.58
4	28,650	14.32	14,454	14.45	14.36
5	43,631	21.82	22,053	22.05	21.87
6	47,225	23.61	23,371	23.37	23.55
7	35,929	17.96	17,937	17.94	17.90
8	18,627	9.31	9,359	9.36	9.40
9	6,416	3.21	3,294	3.29	3.24
10	1,308	0.65	618	0.62	0.66
11	103	0.05	65	0.07	0.06
Total	200,000	100	100,000	100	100

The largest class sets the majority-class baseline accuracy: 23.61% for pqm4 and 23.37% for ref.

Fig. 3.1 shows the three distributions side by side; for every class the three bars are almost indistinguishable.

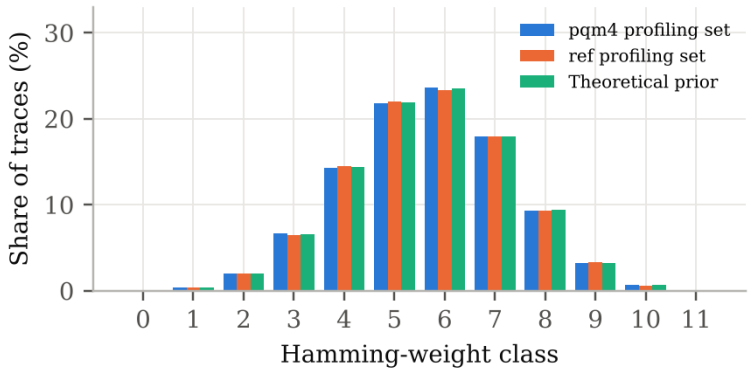


Fig. 3.1 Hamming-Weight Class Shares in the Profiling Sets Compared With the Theoretical Prior

3.5 Functional Requirements

The pipeline has to:

- Stream multi-gigabyte trace and metadata files within the memory available.
- Decode metadata correctly, including the byte-encoded int16 values of ref and the arithmetic shares of the masked implementation.
- Check the fixed-key secret in every attack chunk and against the stored unmasked key, along with the trace-label row pairing.
- Compute per-sample SNR and HW-partitioned Welch t-statistics over the complete profiling set of each implementation.
- Fit a GPD to tail excesses and compute tail scores and tail probabilities.
- Pick points of interest from a score array using minimum-distance exclusion and an optional stop at zero score.
- Split the profiling data into stratified training, validation and test sets, and select points from the training split only.
- Standardise the features train MLP and CNN classifiers, with seeds and calibrate MLP and CNN classifiers.
- Evaluate the models on the test split and complete attack sets. Look at accuracy. Look at confusion matrices. Look at GE. Look at SR. Look at PI. Look at corrected scoring. Evaluate the models on the test split and complete attack sets. Look at accuracy. Look at confusion matrices. Look at

GE. Look at SR. Look at PI. Look at corrected scoring. Evaluate the models, on the test split and complete attack sets. Look at accuracy. Look at confusion matrices. Look at GE. Look at SR. Look at PI. Look at corrected scoring.

3.6 Non-Functional Requirements

- **Memory:** work inside a Colab session by loading chunks of 10,000 traces at a time.
- **Completeness:** use the profiling and attack sets, not parts of them.
- **Reproducibility:** set fixed seeds. Make sure every step saves its results in files like.npz.npy.keras so they can be checked and run again on their own.
- **Auditability:** every number that is shown can be connected to a saved array and a specific notebook cell (Appendix A.3).
- **Robustness to disconnection:** l long processes save their progress model by model. Can continue after a disconnection.
- **Honest reporting:** negative and failed results are shared with their explanation of being ignored.

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 Design Approach / Materials and Methods

One rule guides the whole design: any number used to make a decision must not come from the data that will later be used to judge that decision. Statistics that choose the points for an attack must not see the traces that judge the attack, probabilities used to compare models must be calibrated on data that is not used to report them, and attack results must come from recordings the model has never seen. The first ten stages were built before this rule was fully in place, and Stage 11 was designed to put that right (Section 4.1.3).

4.1.1 System Architecture

Fig. 4.1 shows the three-layer architecture. The data layer loads and verifies traces and metadata and derives HW labels; the assessment layer computes streaming leakage statistics, fits the GPD tail model and selects points of interest; the attack layer builds model inputs, trains and calibrates classifiers and evaluates them on held-out and attack data. The attack layer has two generations: the Stage 7–9 CNN arms (top-50 versus EVT cut-off) and the Stage 11 protocol (training-only selection, three seeds, MLP and CNN, calibration and the full attack sets).

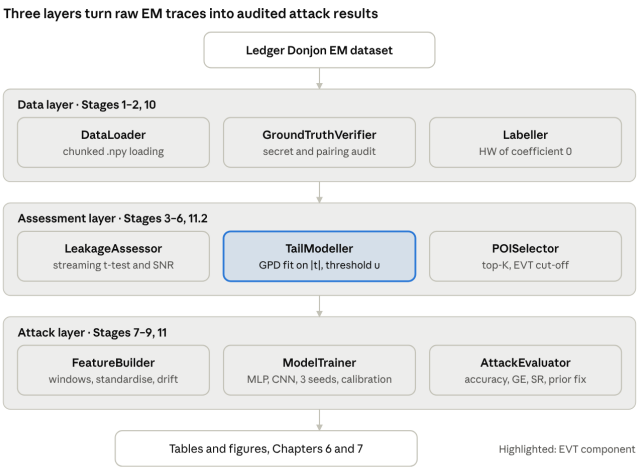


Fig. 4.1 System Architecture of the Complete Pipeline

4.1.2 Streaming Computation of Leakage Statistics

For every sample and class, the t-test and SNR need only the count, sum and sum of squares of the trace values. These sufficient statistics give the means and variances whatever order the traces arrive in, so the pipeline memory-maps one 10,000-trace chunk at a time, converts it to 64-bit floating point, adds its per-class sums to running totals and drops it. Memory use depends on one chunk and the accumulators (12 classes × 20,000 samples), not on the dataset size. The attack-set evaluations in Stages 9 and 11.8 also predict chunk by chunk.

4.1.3 Stage 11 Data Flow

Fig. 4.2 shows the Stage 11 data flow from raw traces to an attack decision. It differs from Stages 4–8 in five respects:

- **Splitting before selection.** The profiling set is split first into stratified training, validation and test parts (80/10/10), and every later statistic uses the training part only.
- **Training-only weighted SNR.** Points are ranked by the class-weighted SNR of Eqn. 1.2 on the training split, and the top K = 50, 100 or 150 are kept.
- **Standardisation.** Each selected point is standardised with the training mean and standard deviation, and the same constants are used for validation, test and attack data; the remaining shift between profiling and attack data is measured as a drift check.
- **Multiple seeds and calibration.** Every configuration is trained with three seeds, and one temperature is fitted to the validation logits.
- **Independent evaluation.** The untouched test split gives the in-session result, and the complete fixed-key attack set gives the cross-session result, scored with prior correction (Eqn. 1.6).

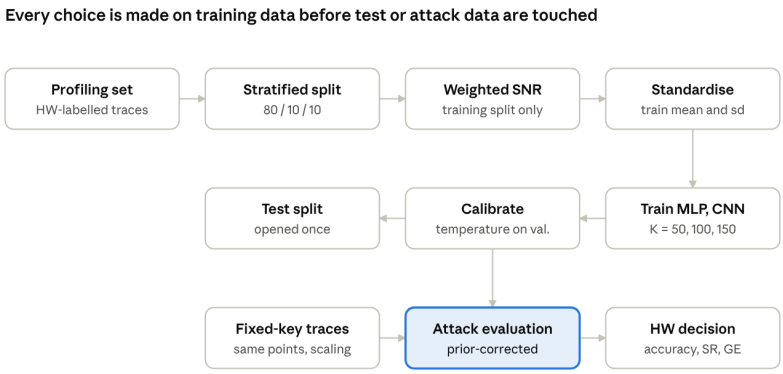


Fig. 4.2 Stage 11 Data Flow From Raw Traces to Attack Decision

4.1.4 Pipeline Modules

The notebook is written as procedural Python, but its functions fall naturally into the modules of Table 4.1.

Module	Main functions	Inputs	Outputs	Stages
DataLoader	Memory-mapped chunk loading, metadata decoding (int16 view for ref), share reconstruction	Raw .npy files	Trace chunks, operands	1–2, 10
GroundTruthVerifier	Fixed-key constancy check, unmasked-key comparison, row-pairing audit	Metadata	Verified secret, pass/fail log	2, 10, 11.0
Labeller	Hamming weight of coefficient 0 of the secret operand	Operands	12-class labels	2, 11.0
LeakageAssessor	Streaming per-class sums; Welch's t; SNR and weighted SNR	Traces, labels	Per-sample |t| and SNR arrays	3, 11.2
TailModeller	GPD fit on |t| excesses, tail score, tail-fraction sensitivity	|t| arrays	Threshold u, ξ, σ, tail scores	5
POISelector	Ranking with minimum-distance exclusion, stop-at-zero, top-K	Score arrays	POI index sets	4, 6, 11.2
FeatureBuilder	POI extraction, training-split standardisation, drift check	Traces, POIs	Model-ready arrays	8, 11.3–11.4
ModelTrainer	MLP/CNN construction, training with callbacks, multi-seed loop, temperature fit	Features, labels	Saved models, temperatures, histories	7–8, 11.5–11.6

Module	Main functions	Inputs	Outputs	Stages
AttackEvaluator	Accuracy, confusion matrix, guessing entropy, success rate, PI, prior correction, null model	Model outputs	Metrics, curves	9, 11.7–11.9

Table 4.1 Pipeline Modules

4.1.5 Model Design

A trace enters a Stage 11 model as K windows of 21 samples, one window around each selected point (Section 6.2.4). The MLP flattens the K × 21 standardised values and feeds them through dense layers of 256, 128 and 64 units with batch normalisation, ReLU and dropout of 0.3, finishing with 12 output logits. The CNN runs one-dimensional filters along each window, with the same filters shared across all K windows: two convolutions of 32 filters of width 5, average pooling, two of 64 filters of width 3, average pooling, one of 128 filters of width 3, average pooling of width 5, a 1×1 convolution down to 32 channels, and dense layers of 256 and 128 units with dropout of 0.4 before the 12 outputs. Table 6.12 gives the parameter counts.

The Stage 7 CNN, in contrast, was kept small on purpose (4,716 parameters) and ended in global average pooling, so that the 50-point and EVT arms used exactly the same architecture. Stage 11 shifted to models because the aim was no longer to compare point sets fairly. The focus changed to achieving the possible accuracy. The models produce logits, which allows temperature scaling to be applied later (softmax of the logits divided by T). No class weights were used. The attack combines probabilities across traces. Removes the class prior explicitly (Eqn. 1.6). Class weights would interfere with the probabilities that this correction relies on so they were avoided.

The two model types handle the window in ways. The MLP assigns its weights to each of the K × 21 inputs. That means the first layer grows as K increases. So a small shift, in the input— one or two samples—ends up using different weights. The CNN on the hand learns a small set of shape detectors once. It then applies these

detectors across every window. Handles small shifts using pooling. Its convolutional part stays fixed at 52,736 parameters no matter what K is (Section 6.2.5).

4.1.6 UML Diagrams

Since this project is a research pipeline and not an interactive application, the use case diagram treats the researcher as the actor, and the component diagram groups the notebook's functions into the modules of Table 4.1. The notation follows OMG UML 2.5.1 (Table 4.2), and the diagrams were drawn with AI assistance (Anthropic, 2026) from the pipeline as implemented.

In the use case diagram (Fig. 4.3), the researcher is the main actor and the Ledger Donjon dataset is an external data source. «include» marks steps that always run inside another use case. When we mention «include» we refer to fitting the GPD during point selection and computing guessing entropy during attack evaluation. «Extend» marks the integrity audit and the session-shift diagnosis. These «extend» items act as checks.

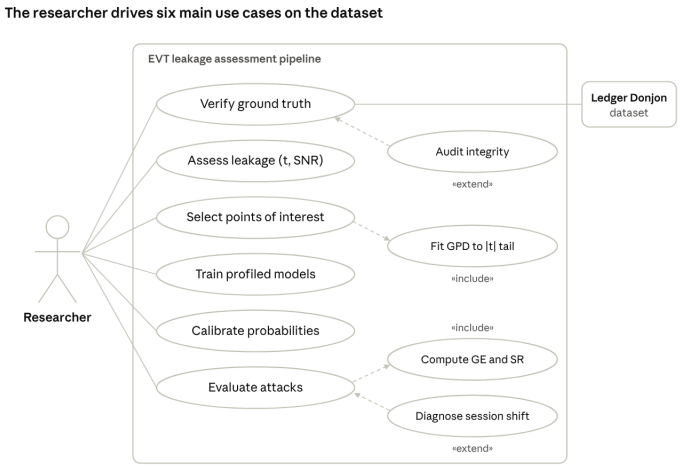


Fig. 4.3 Use Case Diagram

The component diagram (Fig. 4.4) Shows nine modules and their main functions. Solid arrows show how data moves between parts. Dashed arrows show checks. Are drawn from GroundTruthVerifier to DataLoader and, to Labeller.

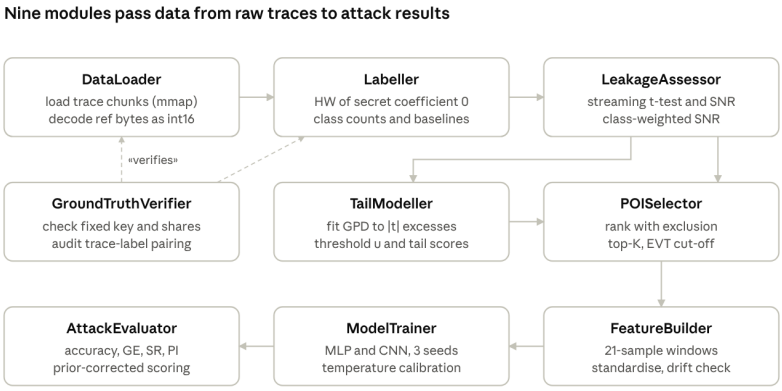


Fig. 4.4 Component (Class) Diagram

4.2 Codes and Standards

Table 4.2 Codes and Standards

Standard or convention	Scope	How it is applied
FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (NIST, 2024)	Definition of ML-KEM	Parameters q = 3329, n = 256, parameter sets (Table 1.1); base multiplication as the target operation
ISO/IEC 17825, testing methods for the mitigation of non-invasive attack classes against cryptographic modules	Leakage assessment practice	Welch's t-test with the $\|t\| > 4.5$ reference level; the secret-dependent (specific) variant is used and labelled as an adaptation
FIPS 140-3 / ISO/IEC 19790	Security requirements for cryptographic modules	Context for why non-invasive leakage is evaluated
OMG UML 2.5.1	Modelling notation	Use case and component diagrams (Figs. 4.3, 4.4)
IEEE 754 double precision	Numerical computation	All streaming sums and statistics accumulated in float64 to avoid precision loss over 200,000 traces
PEP 8 Python style	Code style	Function and variable naming in the notebook
APA 7th edition	Citation style	In-text citations and reference list (Chapter 9)
Dataset licence (LGPL v3.0, as stated in the dataset documentation)	Reuse of data	Data used unmodified for research; source cited

4.3 Constraints, Alternatives and Tradeoffs

4.3.1 Constraints

- **Memory:** the pqm4 profiling set which is about 8 GB as int16 is larger, than session memory therefore every whole-dataset computation must stream.

- **Session time and disconnection:** sessions time out and disconnect long training runs must save results model by model in order to resume.
- **GPU availability:** GPU runtimes are not always free; when a runtime came up without a GPU training had to wait for an L4 session.
- **Fixed data:** the traces were recorded by someone else so no new captures can be made and the pqm4 session shift (Chapter 7) cannot be removed by recording again.
- **Single target coefficient:** labels use coefficient 0 only so the attack recovers one HW class, not a key.

4.3.2 Alternatives and Tradeoffs

Table 4.3 Design Alternatives and Tradeoffs

Decision	Chosen	Alternative(s)	Tradeoff and reason
Leakage model	Hamming weight of coefficient 0 (12 classes)	Identity (3,329 classes); individual bits	HW matches CMOS leakage and gives enough traces per class; it recovers a class, not a value
Test grouping	HW ≤ 6 vs HW > 6 (specific t-test)	Fixed-vs-random TVLA	The dataset has no fixed-vs-random design; the specific test ties detection to the secret directly
POI count (Stages 4–6)	GPD cut-off at the 98th percentile of |t|	Fixed top-50; |t| > 4.5 threshold	Data-driven stop, but depends on the tail fraction (Table 6.3)
POI selection (Stage 11)	Training-only weighted SNR, K = 50/100/150	Whole-set |t|; EVT windows	Removes selection leakage into evaluation; smaller K values dropped by decision as unnecessary
Input scaling	Training-split standardisation	Raw int16 (Stages 7–8)	Stabilises training; exposes session drift as a measurable shift
Models	MLP and CNN, three seeds	Single small CNN; RNNs and transformers	Two families and three seeds separate architecture effects from seed noise within the GPU budget
Class imbalance	No class weights; prior correction at attack time	Class weights; resampling	Keeps probabilities calibrated for combination across traces
Calibration	Temperature scaling	Platt or isotonic scaling; none	One parameter, cannot change the predicted class, standard in SCA practice (Nouraniboosjin & Ganji, 2024)
Attack evaluation	Complete fixed-key sets	First 10,000 traces (Stage 9)	Full sets expose session effects; costs more inference time
pqm4 failure on attack data	Reported as found	Fine-tune on attack data; domain adaptation	Honest reporting; adaptation would mix attack data into training

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

The dissertation ran from 6 July 2026 to 3 November 2026 as a series of numbered stages. Each stage ends with saved outputs that the next one reads, and the stages are grouped under three review milestones. All of them were completed; Table 5.1 lists their tasks and deliverables, and Fig. 5.1 shows the roadmap.

5.1 Stages and Tasks

Table 5.1 Stages, Tasks and Milestones

Stage	Activity	Main tasks	Deliverable	Milestone
1	Dataset acquisition and setup	Download and extract dataset to Drive; mount in Colab; inventory files	Extracted dataset, file inventory	Review 1
2	Data exploration and ground truth	Inspect shapes and dtypes; identify secret operand; decode ref bytes; sweep coefficients	Verified operands and labels	Review 1
3	Baseline leakage assessment	Streaming SNR and Welch t-test on complete profiling sets; masked positive control	|t| and SNR arrays (Table 6.1)	Review 2
4	Conventional POI selection	Top-50 |t| with 20-sample exclusion	Baseline POI sets	Review 2
5	EVT / GPD tail scoring	GPD fit to top 2% of |t|; tail-fraction sensitivity	GPD parameters (Tables 6.2, 6.3)	Review 2
6	EVT-based POI selection	Ranking cut at threshold u	EVT POI sets (Table 6.5)	Review 2
7	CNN architecture design	Small CNN with global average pooling	Model definition	Review 2
8	CNN training	Both arms, pqm4 and ref, full profiling data	Validation accuracies (Table 6.6)	Review 2
9	Attack evaluation	GE, null model, null-corrected advantage on first 10,000 attack traces	Table 6.7	Final

Stage	Activity	Main tasks	Deliverable	Miles tone
10	Integrity audit	Fixed-key checks, row-pairing audit, masked label fix and rebuild	Corrected masked results	Final
11.0–11.1	Data checks and split	Baselines, class counts; stratified 80/10/10 split	Section 6.2.1, Table 6.8	Final
11.2–11.4b	Selection and features	Training-only weighted SNR; feature extraction; standardisation; drift check	Figs. 6.2–6.3, Tables 6.9, 6.11	Final
11.5–11.6	Training and calibration	MLP and CNN, K = 50/100/150, three seeds; temperature scaling	Table 7.1, Fig. 7.1	Final
11.7	Test-split evaluation	Confusion matrices on untouched test split	Figs. 7.2–7.3	Final
11.8–11.8c	Full attack-set evaluation	Accuracy, SR, GE on complete fixed-key sets; pqm4 diagnosis	Tables 7.2, 7.4, Figs. 7.4, 7.6	Final
11.9–11.10	Old versus new models	Stage 8 models on full attack sets; summary figure	Table 7.3	Final
Repor t	Final report and review	Write-up, figures, references	This report	Final

5.2 Milestones

Review 1 settled the problem the base paper and the stage plan. It also completed Stages 1 and 2. By that time the dataset was ready. The secret operand, metadata encoding and labels had all been. Confirmed.

Review 2 (7 October 2026) covered Stages 3 through 8. These stages used the profiling sets. The work included leakage analysis GPD fits, conventional point sets EVT point sets and CNN validation accuracies. The attack evaluation and integrity audit were still pending at that point.

Final review (this report) adds Stages 9 and 10. It also includes the Stage 11 protocol. This covers evaluation, on the attack sets and the diagnosis of the pqm4 failure.

5.3 Changes to the Plan

After Review 2 the plan changed in two decisions. First the extension phases C1–C4 that were sketched in Phase 0 were moved to come after Stage 11. The reason, for moving the extension phases was that none of the extension phases could be judged until a credible leakage-free attack baseline existed; the extension phases remain work. Second Stage 11 was limited to K values of 50, 100 and 150 once the first runs showed change across this range and the smaller K values were dropped.

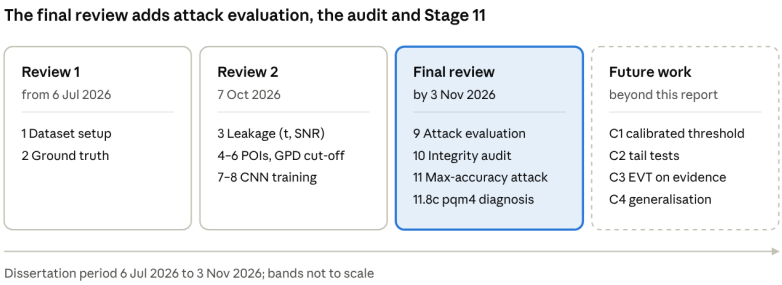


Fig. 5.1 Project Roadmap by Stage and Review

CHAPTER 6

DISSERTATION DEMONSTRATION

6.1 Stages 1–10: Leakage Assessment, EVT Cut-Off and First Attack Evaluation

6.1.1 Stages 1–2: Setup and Ground Truth

The dataset was downloaded to Google Drive then extracted and checked for file counts array shapes and data types. Three findings from this step influenced everything that followed.

First, figuring out which operand is secret. For pqm4 and ref the metadata files mult_a and mult_b do not say. In the fixed-key set the first 200 rows of mult_a were all identical. One unique row. While all 200 rows of mult_b were different. That means a is the fixed secret and b is the changing input.

A sweep of all 256 coefficients for both operands, against the leakage points showed that coefficient 0 of a explains most of the leakage. That coefficient was chosen as the label. The same sweep revealed that the public operand leaks too. At ref sample 6039 the |t| value is 66.9 when looking at b[0]. Only 6.05 when looking at a[0]. So even points picked by a labelled statistic can still carry leakage from the public operand.Second, the ref metadata is stored as raw bytes, 512 uint8 values per 256-coefficient row. Read as little-endian int16, 100% of the values fall in [0, q), compared with only 5.5% under big-endian decoding.

Third, in the masked implementation b is the secret and is split into two arithmetic shares. Adding the shares modulo q gives back the unmasked operand, which was compared with the stored fixed_unmasked_key file. The fixed secret coefficient 0 is 679 (HW 6) for pqm4, 1026 (HW 2) for ref and 1681 (HW 5) for the masked implementation.

6.1.2 Stage 3: Baseline Leakage Assessment

Every profiling trace was labelled with the HW of secret coefficient 0, and two per-sample statistics were streamed over the complete profiling sets: the SNR across the

12 classes, and Welch's t-test between traces with HW ≤ 6 and HW > 6 (Eqn. 1.1). Appendix A.1.10 lists the streaming function.

The t-test reuses the same accumulators, pooling classes 0–6 and 7–11. The results are in Table 6.1.

Table 6.1 Baseline Leakage Results

Implementation	Traces	Max |t| (sample)	Max SNR (sample)	Verdict
pqm4 (unprotected)	200,000	168.39 (183)	1.1352 (183)	Strong leakage
ref (unprotected)	100,000	55.19 (7917)	0.2739 (7917)	Strong leakage
masked (first-order)	100,000	3.20 (1647)	0.0428	Below 4.5; no significant first-order leakage of the secret

To make sure the masked null result was not down to an insensitive test, the 100,000 masked traces were relabelled using the HW of share b0 alone. The maximum |t| jumped to 262.83, with a maximum SNR of 1.083 at sample 516. The share leaks strongly and the secret does not, which is exactly how a working first-order masking scheme should behave.

6.1.3 Stage 4: Conventional Point-of-Interest Selection

The conventional arm takes the 50 highest-|t| points and blocks out 20 samples on either side of each pick, so that one leakage peak cannot supply several almost identical points (Appendix A.1.1).

6.1.4 Stage 5: EVT/GPD Tail Scoring

The absolute t-statistics of all samples in a trace were treated as a sample from an unknown distribution. The threshold u was set at their 98th percentile (tail fraction 0.02), the excesses above u were fitted with a GPD whose location was fixed at zero, and each sample above u was scored by $-\log_{10}$ of its GPD survival probability. Samples at or below u score zero.

```
def evt_tail_score(stat_array, tail_fraction=0.02):
    abs_stat = np.abs(stat_array)
    u = np.quantile(abs_stat, 1 - tail_fraction)
    excesses = abs_stat[abs_stat > u] - u
    shape, loc, scale = genpareto.fit(excesses, floc=0)
    scores = np.zeros_like(abs_stat, dtype=np.float64)
    mask = abs_stat > u
    sf = genpareto.sf(abs_stat[mask] - u, shape, loc=0, scale=scale)
    scores[mask] = -np.log10(np.clip(sf, 1e-300, 1.0))
    return scores, u, (shape, loc, scale)
```

Table 6.2 gives the fitted parameters on the complete profiling data.

Table 6.2 GPD Fits at Tail Fraction 0.02

Implementation	Threshold u (98th percentile of |t|)	Shape ξ	Scale σ	Max EVT score (sample)
pqm4	9.676	0.564	13.051	1.588 (183)
ref	9.964	0.081	8.764	1.873 (7917)

The tail fraction is a design choice, and it matters. On an earlier 20,000-trace subset (Table 6.3) the fitted shape is not stable across tail fractions: for pqm4 it moves between 0.59 and 1.16, and for both implementations ξ rises from the 0.02 fraction to a peak at 0.10 and then falls. The value 0.02 controls how much of the data is considered extreme. It limits the fit, to the extreme 2% of samples. This number was not chosen for stability. The number of EVT points depends directly on this value.

Table 6.3 Tail-Fraction Sensitivity

Tail fraction	pqm4 u	pqm4 ξ	ref u	ref ξ
0.02	3.375	+0.594	4.386	+0.078
0.05	1.817	+1.051	2.462	+0.520
0.10	1.443	+1.163	1.829	+0.726
0.15	1.289	+1.032	1.513	+0.620
0.20	1.171	+0.857	1.325	+0.599

The shape of the distribution also varies between the two leaking implementations. Pqm4 shows a heavy tail (ξ = 0.564) while ref, which also leaks significantly is close to exponential (ξ = 0.081). Because of this difference the value of ξ is. Not used as evidence for or against leakage.

Two important points, about the score must be made clearly. The generalized Pareto distribution survival function always decreases, above the threshold u the score maintains the exact ranking of |t|. The fitted shape and scale parameters do not affect which data points are selected—only u has that influence.

Additionally neighboring samples are highly correlated,. The model assumes that excesses are independent. No declustering was done to account for this dependence.

Because u is the 98th percentile, 400 of the 20,000 pqm4 samples and 260 of the 13,000 ref samples lie above it and go into the fit. Table 6.4 works through the score by hand for the strongest sample. For pqm4 the excess is y = 168.39 − 9.676 = 158.71, so $1 + \xi y/\sigma = 1 + 0.564 \times 158.71 / 13.051 = 7.859$. The survival probability is then 7.859^(−1/0.564) = 0.0259, and the score is $-\log_{10}(0.0259) = 1.588$, the maximum in Table 6.2.

Table 6.4 Worked Example of the EVT Score for the Strongest Sample

Quantity	**pqm4**	**ref**
Maximum |t| (sample)	168.39 (183)	55.19 (7917)
Threshold u (98th percentile)	9.676	9.964
Excess y = |t| − u	158.71	45.23
Shape ξ / scale σ	0.564 / 13.051	0.081 / 8.764
$1 + \xi y/\sigma$	7.859	1.418
GPD survival probability	0.0259	0.0134
Score = $-\log_{10}$(survival)	1.588	1.873

The survival probability depends on exceeding u so the survival probability tells us how extreme a sample is within the two percent above the threshold.This is why even the highest scores remain below two.Refs sample scores higher than pqm4s (1.873 versus 1.588) even though refs |t| is about three times smaller because refs near-exponential tail thins out faster: an excess of 45 is rarer under the ref fit than an excess of 159 is, under the pqm4 fit. Fig. 6.1 shows both fitted survival curves on a

log scale next to an exponential reference; on this scale an exponential tail is a straight line and a heavy tail bends upward.

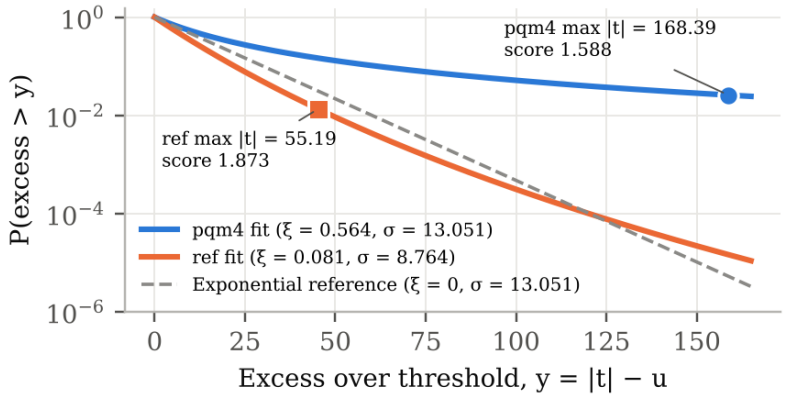


Fig. 6.1 Fitted GPD Survival Curves of the |t| Tail, pqm4 and ref

6.1.5 Stage 6: EVT-Based Point Selection

The EVT arm runs the same selection on the EVT score and stops as soon as the next candidate scores zero. Since the score keeps the |t| ranking, the EVT set is always the start of the conventional list, cut off at u (Table 6.5).

Table 6.5 Conventional Versus EVT-Based Point Selection

Implementation	Conventional POIs	EVT POIs	Reduction	Overlap with conventional
pqm4	50	27	46%	27/27 (first 27 of the list)
ref	50	15	70%	15/15 (first 15 of the list)
masked	50	50	0%	50/50

For the masked implementation the cut-off removed nothing. With no real first-order leakage, the selected |t| values do not stand out from the rest.

The 400 pqm4 and 260 ref samples above u give only 27 and 15 points, because the 20-sample exclusion window lets each leakage peak contribute just one point. Once 27 (pqm4) or 15 (ref) points have been chosen, every remaining sample above u falls inside the window of a point already selected.

6.1.6 Stages 7–8: CNN Design and Training

Both arms used the same one-dimensional CNN: Conv1D (16 filters, kernel 3), batch normalisation, max pooling (2), Conv1D (32, kernel 3), batch normalisation, global average pooling, Dense (64), dropout 0.3 and a 12-way softmax. Global average pooling keeps it at 4,716 trainable parameters however many points go in, which keeps the 50-versus-27 and 50-versus-15 comparisons fair. Training used Adam (learning rate 0.001), sparse categorical cross-entropy, batch size 256, a stratified 90/10 train/validation split and up to 50 epochs, with early stopping (patience 10) and checkpointing on validation accuracy. The inputs were the raw int16 amplitudes at the selected points (Table 6.6).

Table 6.6 Stage 8 CNN Validation Accuracy

Implementation	Arm	POIs	Validation accuracy
pqm4	Conventional	50	31.42%
pqm4	EVT	27	31.22%
ref	Conventional	50	50.45%
ref	EVT	15	46.86%
masked	Both	50	about 23.5%

The EVT arms lost 0.2 (pqm4) and 3.6 (ref) percentage points of validation accuracy while using 46% and 70% fewer points. The masked models stayed at about 23.5%, essentially the majority-class rate, which fits with the absence of first-order leakage. Each configuration was trained only once, and pqm4 validation accuracy swung between about 21% and 31% from epoch to epoch, so the 0.2-point pqm4 gap is within run-to-run noise.

6.1.7 Stage 9: First Attack Evaluation

The Stage 8 models were tested on the first 10,000 fixed-key attack traces. Guessing entropy, found by ranking the 12 classes by cumulative log-probability over random orderings of the traces, saturated for pqm4. The true class (HW 6) is also the most common class in training, so even a null model that just outputs the training class frequencies ranks it first, and rank on its own cannot show whether the models learned anything (Rousselot et al., 2026).

A null-corrected advantage was computed to get around this. It takes the per-trace mean log-likelihood advantage of the true class over its strongest competitor, for both the model and the null model, and reports the difference Δ = model advantage − null advantage (Table 6.7). A positive Δ means the traces pushed the model toward the true class further than the class frequencies alone would.

Table 6.7 Stage 9 Attack Evaluation (First 10,000 Traces)

Model	Competitor class	Null advantage	Model advantage	Δ (model − null)
pqm4 conventional	5	0.0792	−0.3742	−0.4534
pqm4 EVT	5	0.0792	−0.4276	−0.5068
ref conventional	6	−2.4578	4.8998	7.3576
ref EVT	6	−2.4578	3.9134	6.3712
masked conventional (after Stage 10 fix)	6	—	—	−0.0197
masked EVT (after Stage 10 fix)	6	—	—	+0.0155

The ref models clearly have information about the fixed secret: both arms move strongly toward the class (HW 2) well past the null model. The pqm4 models move away from the class (HW 6) and move toward HW 5 so their apparent rank-0 result came from the prior and not from the traces. This was the hint of the problem diagnosed in Stage 11.8c. The masked values sit close, to zero as expected when there is no first-order leakage.

6.1.8 Stage 10: Integrity Audit

Stage 10 reviewed the ground truth and the trace-label pairing for every stage.The fixed-key secret remained the same in every chunk.For the implementation the fixed-key secret matched both the share reconstruction and the stored unmasked key.The row pairing of trace and metadata chunks was checked for all three implementations.The audit discovered that the labels in the masked runs did not line up with trace-label pairing.The labels were rebuilt from paired share files.The masked results of Stages 4–9 were. The masked values, in Tables 6.5–6.7 are the corrected ones.The unprotected pipelines passed without changes.

6.2 Stage 11: Maximum-Accuracy Profiled Attack

Stages 7–9 compared the two point sets but there were three weak spots: points were picked from the entire profiling set even traces that were later used for validation; the inputs were raw amplitude values; and each model was trained only once. Stage 11 rebuilt the attack under the leakage-free protocol of Section 4.1.3, aiming for the highest accuracy with the fewest points. It covers pqm4 and ref, since the masked implementation has no first-order leakage to exploit (Table 6.1).

6.2.1 Stage 11.0: Data Checks

Before splitting, the labels and secrets were checked again and the baselines worked out. Both profiling sets contain all 12 classes, in proportions within 0.1 percentage point of the prior (Table 3.4). The majority class, HW 6, gives baseline accuracies of 23.61% (pqm4) and 23.37% (ref), compared with 8.33% for uniform guessing, and the smallest class, HW 0, has 68 pqm4 and 24 ref traces. The fixed secret coefficient 0, 679 (HW 6) for pqm4 and 1026 (HW 2) for ref, is the same in every attack chunk, and the ref metadata is read as uint8 bytes viewed as int16.

The pqm4 secret falls in the majority class, so any model that leans toward HW 6 will seem to succeed. The ref secret, HW 2, has a prior probability of only 1.98%, so a model can rank it first only by actually reading the traces.

6.2.2 Stage 11.1: Stratified Split

The profiling set was split 80/10/10 into training, validation and test parts, stratified so that each part keeps the class proportions, using a fixed random seed (Table 6.8).

No decision in Stage 11 used the test split; it was opened once, in Stage 11.7.

Implementation	Training	Validation	Test	Smallest class (HW 0)
pqm4	160,000	20,000	20,000	68 traces, split in the same 80/10/10 proportion
ref	80,002	9,999	9,999	24 traces: 20 / 2 / 2

Table 6.8 Stage 11.1 Stratified Split

The ref split sizes are not exact multiples of 10% because stratification rounds within each class. HW 0 has only two ref traces in each of the validation and test parts, so per-class results for the extreme classes are not interpreted.

6.2.3 Stage 11.2: Training-Only Point Selection

Points were ranked by the class-weighted SNR of Eqn. 1.2 on the training split, with weights equal to the class proportions. Without weighting, the rare classes HW 0, 1, 10 and 11, whose means rest on a few dozen traces, count as much as HW 6, and their noisy means push up the SNR at random points. The computation is shown below.

```
def weighted_snr(X, y, n_classes=12):
    w = np.bincount(y, minlength=n_classes) / len(y)
    mu = np.stack([X[y == c].mean(0) for c in range(n_classes)])
    var = np.stack([X[y == c].var(0) for c in range(n_classes)])
    grand = (w[:, None] * mu).sum(0)
    signal = (w[:, None] * (mu - grand) ** 2).sum(0)
    noise = (w[:, None] * var).sum(0)
    return signal / noise
```

The top 150 points were saved in ranked order, so K = 50, 100 and 150 are nested slices of the same list. Table 6.9 summarises the selection, and Figs. 6.2 and 6.3 show the SNR along the trace with the chosen points marked.

Quantity	pqm4	ref
Maximum weighted SNR (sample)	0.2526 (183)	0.0622 (7917)
Top-10 points	183, 731, 1119, 695, 241, 768, 207, 1630, 34, 3558	7917, 5683, 7957, 5722, 7996, 6039, 8684, 5779, 8280, 6449
SNR of 100th point	0.00012	0.00034
SNR of 150th point	0.00011	0.00030
Approximate noise floor, 11/N_train	0.00007	0.00014
Overlap with unweighted-SNR top 150	27 / 150	24 / 150

Table 6.9 Stage 11.2 Point Selection Summary

Three things stand out. The strongest points match Stage 3: samples 183 (pqm4) and 7917 (ref) were also the maximum-|t| points. The SNR drops quickly, and by the 100th point it is within a factor of two or three of the noise floor (about (C − 1)/N, Section 1.3.5), so points beyond the first few dozen add little information and some noise. The weighted and unweighted selections also share only 27 and 24 of 150 points, which confirms that past the top few points the unweighted SNR is driven mostly by noise from the rare classes.

The size of the weighting effect can be estimated from the class counts alone. Even with no leakage, class means differ from the overall mean through sampling error, so a pure-noise sample still has a non-zero SNR. For the weighted SNR its expected value is approximately

$$\mathrm{E}[\mathrm{SNR}_w] \approx \frac{C-1}{N} \qquad \text{(Eqn. 6.1)}$$

while for the unweighted SNR of Stage 3 (the variance of the 12 class means divided by the mean within-class variance) it is approximately

$$\mathrm{E}[\mathrm{SNR}_u] \approx \frac{C-1}{C^2}\sum_c \frac{1}{N_c} \qquad \text{(Eqn. 6.2)}$$

where C = 12 is the number of classes, N the number of training traces and Nc the number in class c. Table 6.10 evaluates both using the training class counts, which are about 80% of those in Table 3.4.

Table 6.10 Expected SNR of a Non-Leaking Sample (Noise Floor), Training Split

Quantity	pqm4	ref
Training traces N	160,000	80,002
Training traces in the smallest class (HW 0)	≈ 54	20
Weighted noise floor, Eqn. 6.1	≈ 0.00007	≈ 0.00014
Unweighted noise floor, Eqn. 6.2	≈ 0.0026	≈ 0.0060
Ratio, unweighted / weighted	≈ 38×	≈ 44×

Quantity	pqm4	ref
Share of the unweighted floor due to HW 0 and HW 11	≈ 90%	≈ 91%

The unweighted floor is about 40 times the weighted one, and about 90% of it comes from HW 0 and HW 11. The weighted SNR of the 100th and 150th points (0.00011–0.00034, Table 6.9) is far below the unweighted floor, so with the unweighted statistic these points would be indistinguishable from noise samples whose HW 0 or HW 11 mean happens to drift by chance. This fits the small overlap between the two top-150 lists.

The ref top-10 includes sample 6039, which leaks the public operand b much more strongly than the secret (Section 6.1.1). Points like this can slip into a secret-labelled selection through partial correlation with the label.

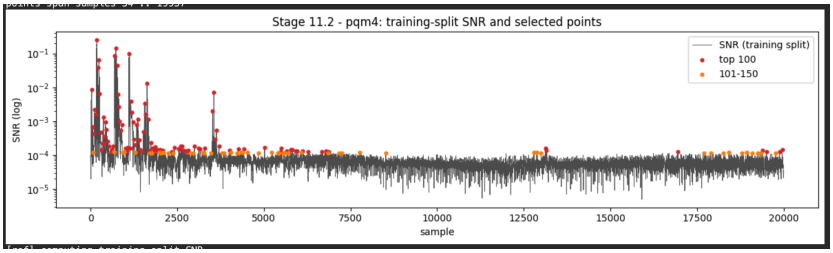


Fig. 6.2 Training-Split SNR and Selected Points, pqm4

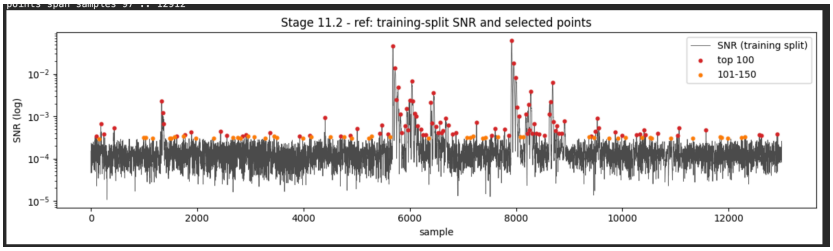


Fig. 6.3 Training-Split SNR and Selected Points, ref

6.2.4 Stages 11.3–11.4: Features, Standardisation and Drift Check

For each K, a 21-sample window around each selected point was cut from every training, validation, test and attack trace, giving K × 21 inputs (1,050 at K = 50). Each input was standardised with the training mean and standard deviation, and the same constants were used on all other data, so a feature that follows the training distribution ends up with mean near 0 and standard deviation near 1.

The drift check measures how far each data set moves away from this reference, using the median over points of the absolute mean of the standardised feature (Table 6.11).

Table 6.11 Stage 11.4 Profiling-Versus-Attack Drift Check

Quantity	pqm4	ref
Validation median |mean z|	0.0058	0.0078
Attack median |mean z|	0.120	0.0288
Attack / validation ratio	about 21×	about 4×
Median raw shift (attack − profiling)	−0.138	+0.087
Fraction of points shifted upward	0.45	0.62
Shift correlation (as reported by the check)	−0.067	−0.016
Spread ratio (attack std / profiling std)	0.9835	1.0019
Median |shift| in leaky windows vs noise windows	0.144 vs 0.150	0.0447 vs 0.0224

The validation data are where they should be with offsets below 0.01 standard deviations. The ref attack data are off by 0.03, which is small compared to the class separations that the ref model depends on. The pqm4 attack data are off by 0.12, which's 21 times the validation level. The shift is just as large in windows without leakage as it is in ones—0.150 versus 0.144. The spread stays the same with a ratio of 0.98. A broad shift in level with the noise pattern is what you would expect from a change in recording conditions between sessions not from a change, in the leakage itself. Stage 11.8c later confirmed that this shift alters the pqm4 model's decisions.

6.2.5 Stage 11.5: Model Training

Two model families were trained for each implementation and each K (Section 4.1.5). Table 6.12 lists the architectures and parameter counts, and Table 6.13 the training settings.

Table 6.12 Stage 11 Model Architectures

Model	Layers	Parameters K = 50	K = 100	K = 150
MLP	Flatten → Dense 256 → Dense 128 (each with batch normalisation, ReLU, dropout 0.3) → Dense 64 → Dense 12 (logits)	312,524	581,324	850,124
CNN	Conv 1×5 (32) → Conv 1×5 (32) → AvgPool → Conv 1×3 (64) → Conv 1×3 (64) → AvgPool → Conv 1×3 (128) → AvgPool 1×5 → Conv 1×1 (32) → Dense 256 → Dense 128 (dropout 0.4) → Dense 12 (logits)	499,980	909,580	1,319,180

The parameter counts can be worked out layer by layer from the K × 21 input. In the MLP at K = 50, the first dense layer on the 1,050 flattened inputs holds 269,056 of the 312,524 parameters (86%), because it gives every input its own weights; Dense 128 adds 32,896, Dense 64 adds 8,256, the 12 logits 780 and batch normalisation 1,536. In the CNN, the convolutional part that learns the leakage shape costs only 52,736 parameters, shared by all windows and independent of K. Most of its 499,980 parameters sit in the 256-unit dense layer on the 50 × 32 = 1,600 window features (409,856), with 32,896 in Dense 128, 1,548 in the logits and 2,944 in batch normalisation. For larger K only the first dense layer grows, by 5,376 parameters per point in the MLP (21 × 256) and 8,192 in the CNN (32 × 256), which gives the K = 100 and 150 totals.

Table 6.13 Stage 11 Training Settings

Setting	Value
Optimiser	Adam, initial learning rate 1×10^{-3}
Loss	Sparse categorical cross-entropy on logits
Batch size	512
Maximum epochs	100
Early stopping	On validation loss, patience 10, best weights restored
Learning-rate schedule	ReduceLROnPlateau, factor 0.5, patience 4
Regularisation	L2 weight decay 1×10^{-4}; dropout as in Table 6.12

Setting	Value
Class weights	None (prior handled at attack time)
Seeds	Three per configuration (pqm4 MLP, pqm4 CNN K = 50, ref CNN); one for pqm4 CNN K = 100/150 and ref MLP
Hardware	NVIDIA L4 GPU
Inference batch	1,024 (fallback 256)

The code that builds the MLP is listed in Appendix A.1.11; the CNN is built the same way using the convolutional stack of Table 6.12.

The number of seeds varies for practical reasons. The pqm4 CNN at K = 50 did no better than the MLP, so the CNN was run only once at K = 100 and 150, and the ref MLP was clearly behind the ref CNN after one seed, so the GPU time went to the CNN. Each model's history, weights and validation metrics were saved to Drive as soon as it finished, so that a dropped session could carry on.

6.2.6 Stage 11.6: Calibration

For each trained model, one temperature T was fitted on the validation split by minimising the negative log-likelihood of softmax(logits / T). Temperature scaling never changes which class scores highest, so accuracy stays the same; it only adjusts how confident the probabilities are. The calibrated probabilities were used for perceived information, prior-corrected scoring and success rate. A one-off TensorFlow retracing warning appeared and did no harm.

6.2.7 Stages 11.7–11.10: Evaluation

The remaining stages evaluate the models, and Chapter 7 presents what they found.

- **11.7, test split:** accuracy and confusion matrices on the untouched 10% test split (Figs. 7.2, 7.3).
- **11.8, full attack sets:** calibrated, prior-corrected predictions for all 200,000 pqm4 and 100,000 ref attack traces, worked out in chunks; attack accuracy, success rate at 1, 10 and 100 traces, and guessing entropy (Table 7.2,

Fig. 7.4). A cuDNN out-of-memory error at prediction batch 8,192 (ref CNN, K = 150) was solved with batches of 1,024 and a fallback to 256.

- **11.8c, pqm4 diagnosis:** the expected HW predicted for profiling traces of the secret's class, for profiling traces with exactly the secret's value, by a bit-level model, and for the attack traces (Table 7.4, Fig. 7.6).
- **11.9, old versus new:** the Stage 8 models tested on the full attack sets with the same metrics (Table 7.3), with a summary figure of the comparison produced in Stage 11.10.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Validation Accuracy

Table 7.1 gives the validation accuracy of every Stage 11 configuration, as mean and standard deviation over three seeds where three were run; Fig. 7.1 plots accuracy against the number of points K.

Table 7.1 Validation Accuracy Over Three Seeds

Implementation and model	K = 50	K = 100	K = 150	Majority baseline
pqm4 MLP	39.61 ± 0.13% (val. loss 1.4325)	39.12 ± 0.21% (1.4477)	39.13 ± 0.05% (1.4491)	23.61%
pqm4 CNN	39.12 ± 0.26% (1.4669)	38.71% (1 seed)	37.87% (1 seed)	23.61%
ref CNN	94.13 ± 0.29% (0.2574)	94.54 ± 0.46% (0.2613)	94.77 ± 0.18% (0.2675)	23.37%
ref MLP (1 seed)	89.93%	88.59%	86.49%	23.37%

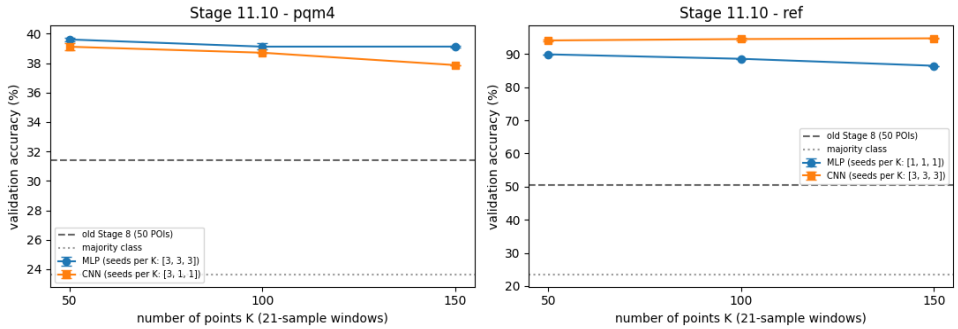


Fig. 7.1 Validation Accuracy Versus Number of Points

7.1.1 ref

The ref CNN reaches 94.1–94.8% validation accuracy, four times the majority baseline, and different seeds vary by less than half a percentage point. K = 150 is best by accuracy (94.77%) and K = 50 by validation loss (0.2574). The gaps between K values, 0.2 to 0.6 points, are the same size as the seed spread, so 50 points are enough.

The CNN beats the MLP by 4 to 8 points, and the gap widens with K because the MLP gets worse as points are added (89.9% to 86.5%) while the CNN improves

slightly. The ref leakage is spread over neighbouring samples across several clock cycles (Table 6.9 lists points such as 7917, 7957 and 7996, about 40 samples apart). A convolution can combine such patterns with shared weights, whereas the MLP has to learn new weights for every added point and over-fits more as K grows.

7.1.2 pqm4

The pqm4 models reach 39.1–39.6% validation accuracy, 1.68 times the majority baseline and 4.7 times uniform guessing. The leakage is real and the models learn it, but most pqm4 traces cannot be classified on their own. More points do not help (MLP 39.6, 39.1, 39.1%; CNN 39.1, 38.7, 37.9%), and the MLP's validation loss creeps up (1.4325 to 1.4491), so the extra points bring noise rather than information, in line with the fast drop of the weighted SNR (Table 6.9). Here the MLP does at least as well as the CNN, since pqm4 has one dominant leakage point (sample 183, SNR 0.25 against 0.06 for the best ref point) and little local structure for a convolution to use.

7.1.3 Comparison With Stage 8

Compared with the Stage 8 CNNs on 50 points (Table 6.6), the Stage 11 models gain 8.2 points on pqm4 (31.42% to 39.61%) and 43.7 points on ref (50.45% to 94.13% at K = 50). Three changes played a part: standardised inputs, ranking points by the class-weighted SNR instead of a median-split |t|, and larger models trained longer with learning-rate reduction. No ablation was run to separate them, but the ref jump is too large to come from model size alone. The HW $\leq$ 6 versus HW > 6 split cannot tell, for example, HW 2 from HW 5, whereas the weighted SNR ranks points by how well they separate all 12 classes.

So the answer to the “how few points” question is K = 50 for both implementations. Neither gets meaningfully better beyond it, and for pqm4 more points make the model slightly worse.

7.2 Test-Split Results

The untouched test split was opened once, after every choice of model and K had been made. Figs. 7.2 and 7.3 show the row-normalised confusion matrices of the ref CNN and the pqm4 MLP at K = 50.

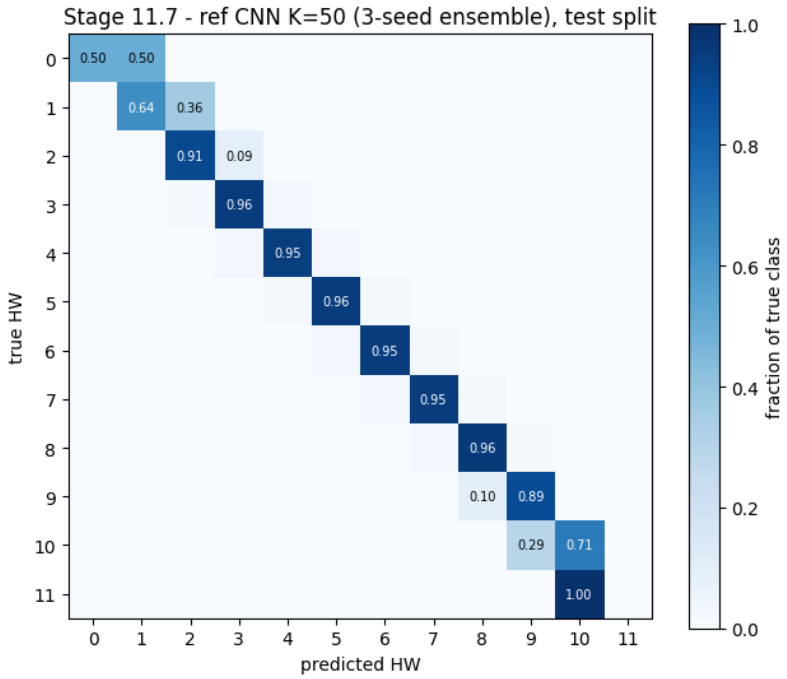


Fig. 7.2 Confusion Matrix, ref CNN K = 50 (Three-Seed Ensemble), Test Split

For ref the diagonal dominates. HW 2 to 9 are classified correctly 89–96% of the time, and nearly every error lands on a neighbouring class (±1). HW 0, 1, 10 and 11 have too few test traces (as few as 2) for their rows to mean much. On test traces of the attacked class, HW 2, accuracy is 90.5% (K = 50) and 91.5% (K = 100 and 150).

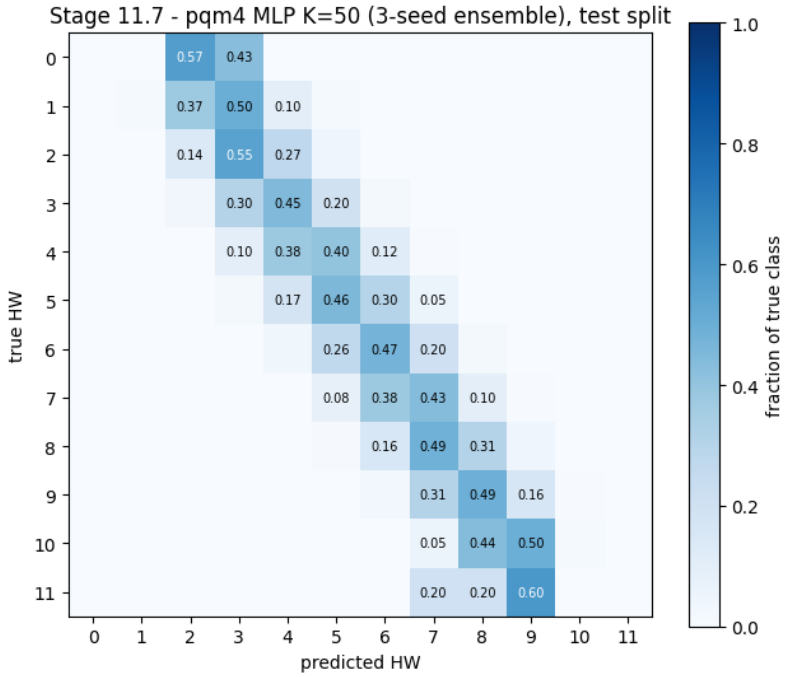


Fig. 7.3 Confusion Matrix, pqm4 MLP K = 50 (Three-Seed Ensemble), Test Split

For pqm4, HW 5 to 7 are classified correctly 43–47% of the time and HW 4 38%, HW 3 and HW 8 about 0.30, and the remaining classes 0.16 or less. Errors fall on neighbouring classes and predictions are pulled toward the middle (for example, 0.55 of HW 2 traces are predicted as HW 3 and 0.49 of HW 8 traces as HW 7). This is what a classifier with weak evidence and an unequal prior is expected to do: when a trace says little, the model falls back on the common classes. On test traces of the attacked class, HW 6, accuracy is 47.4%.

The test results agree with validation, so the validation split was not over-used. They also set the benchmark for the attack sets: on traces from the profiling session, the attacked class is classified correctly about 91% of the time for ref and 47% for pqm4.

7.3 Attack-Set Results

The attack sets were recorded with one fixed secret, in a different session from the profiling data. They are the real test of a profiled attack, where the attacker trains on their own device and has to succeed on the victim's recordings.

7.3.1 New Models on the Full Attack Sets

Table 7.2 gives the Stage 11 results on all 100,000 ref and 200,000 pqm4 attack traces. Attack accuracy is the fraction of single traces whose most probable class is the secret's class; SR at n traces is the fraction of disjoint groups of n traces whose combined, prior-corrected score (Eqn. 1.6) ranks the true class first.

Table 7.2 Full Attack-Set Results, New Models

Model	Attack accuracy	Same-class test accuracy	SR, 1 trace	SR, 10 traces	SR, 100 traces	Traces until GE < 0.5	Final decision (true class)
ref CNN K = 50	94.65%	90.5%	0.892	1.000	1.000	1	HW 2 (HW 2)
ref CNN K = 100	94.15%	91.5%	0.857	—	—	—	HW 2 (HW 2)
ref CNN K = 150	94.42%	91.5%	0.852	—	—	—	HW 2 (HW 2)
pqm4 MLP K = 50	32.38%	47.4%	0.237	0.05	0	Not reached	HW 5 (HW 6)
pqm4 CNN K = 50	32.75%	—	—	—	—	Not reached	HW 5 (HW 6)
pqm4 MLP K = 100	33.30%	—	—	—	—	Not reached	HW 5 (HW 6)
pqm4 MLP K = 150	31.30%	—	—	—	—	Not reached	HW 5 (HW 6)

The ref attack works completely. Accuracy on the attack traces (94.65%) is as high as on validation data (94.13%), a single trace identifies the secret's HW class 89% of the time and ten traces always do, and the prior-corrected guessing entropy is already below 0.5 after one trace and reaches 0 within about five traces (Fig. 7.4). The small ref session drift (Table 6.11) does not hurt the attack.

The pqm4 attack fails. Single-trace accuracy drops from 47.4% on test traces of the same class to 32.4% on the attack traces, and the success rate falls as more traces are added (0.237 at one trace, 0.05 at ten, zero at 100 and beyond), which means the attack is steadily biased toward a wrong answer. Every chunk of the attack set, and every pqm4 model, settles on HW 5 instead of HW 6, with the true class in second place (rank 1). With prior-corrected scoring the ranking locks onto this wrong decision within about 3,000 traces at K = 50 and about 50,000 at K = 150.

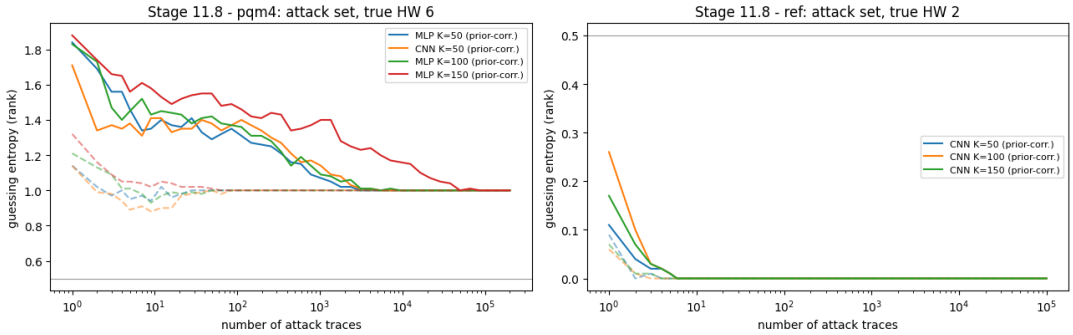


Fig. 7.4 Guessing Entropy on the Full Attack Sets

7.3.2 Old Versus New Models

To see what Stage 11 changed, the Stage 8 models (raw inputs, |t|-selected points, small CNN) were run on the same full attack sets with the same metrics (Table 7.3). “Plain” uses the raw sum of log-probabilities (Eqn. 1.4), and “corrected” removes the prior (Eqn. 1.6).

Table 7.3 Old (Stage 8) Versus New (Stage 11) Models on the Full Attack Set

Model	Attack accuracy	Final rank of true class (plain / corrected)	Decision (plain / corrected)	Traces until GE < 0.5 (corrected)	SR at 1 / 10 / 100 traces (corrected)
ref old, top-50	26.38%	2 / 0	HW 3 / HW 2	4	0.458 / 0.897 / 1.000
ref old, EVT-15	26.86%	2 / 0	HW 3 / HW 2	4	0.413 / 0.909 / 1.000
ref new, CNN K = 50	94.65%	0 / 0	HW 2 / HW 2	1	0.892 / 1.000 / 1.000

Model	Attack accuracy	Final rank of true class (plain / corrected)	Decision (plain / corrected)	Traces until GE < 0.5 (corrected)	SR at 1 / 10 / 100 traces (corrected)
pqm4 old, top-50	12.17%	2 / 4	HW 5 / HW 4	Not reached	0.053 / — / —
pqm4 old, EVT-27	10.64%	2 / 6	HW 5 / HW 0	Not reached	0.049 / — / —
pqm4 new, MLP K = 50	32.38%	1 / 1	HW 5 / HW 5	Not reached	0.237 / 0.05 / 0

Four points stand out.

- **The ref improvement is large and real on attack data.** Attack accuracy climbs from about 26% to 94.7%, and the number of traces needed for the corrected guessing entropy to stay below 0.5 drops from 4 to 1.
- **Prior correction rescued the old ref models.** With plain scoring both pick HW 3, a common class, and put the true HW 2 third; with the prior removed they pick HW 2 within 4 traces. I had learned information, about HW 2 but the prior hid HW 2 which is the failure mode described by Rousselot et al. (2026).
- **The EVT cut-off cost nothing on ref attack data.** With 15 points of 50 the old EVT model still requires the same 4 traces. It even performs better at 10 traces with a success rate of 0.909 compared to 0.897. On pqm4 both old arms fail. The EVT arms corrected decision becomes degenerate showing a hardware value of 0.
- **Stage 11 moved pqm4 closer but not to success.** The true class moves up from rank 2–6 to rank 1 and attack accuracy almost triples, but the decision is still HW 5.

7.3.3 Diagnosis of the pqm4 Failure

Three explanations were possible for the pqm4 failure: the model is too weak, the secret value 679 is unusually hard even within the profiling session, or the attack recordings differ from the profiling recordings. Stage 11.8c told them apart using the model's expected HW, the probability-weighted average of the predicted classes, for four groups of traces (Table 7.4).

Table 7.4 pqm4 Diagnosis: Expected Hamming Weight by Session

Trace group	pqm4: expected HW (n)	ref: expected HW (n)
Held-out profiling traces of the secret's class	5.890 (9,444 traces of HW 6)	2.086 (traces of HW 2)
Held-out profiling traces with exactly the secret's value	6.151 (12 traces of value 679)	1.994 (5 traces of value 1026)
Bit-level model prediction for the secret's value	5.915 ($R^2 = 0.683$; value at the 57th percentile)	2.067 ($R^2 = 0.984$)
Attack traces (fixed secret)	5.273	2.064

For ref, all four numbers agree to within 0.1, so the model reads the attack traces just as it reads profiling traces of the same value. For pqm4, the three profiling-session numbers agree with each other (5.89, 6.15, 5.92): profiling traces of value 679 are read as about HW 6, and the bit-level model shows that 679 is an ordinary value for this model, at the 57th percentile of difficulty. The attack traces, though, are read as HW 5.27, a drop of 0.6 to 0.9 HW units that nothing about the value explains. What is left is a difference between the recording sessions, which matches the drift check (Table 6.11): a broad level shift of 0.12 standard deviations across leaky and non-leaky windows alike, with the spread unchanged.

Fig. 7.5 plots the four values for each implementation against the true HW of the secret.

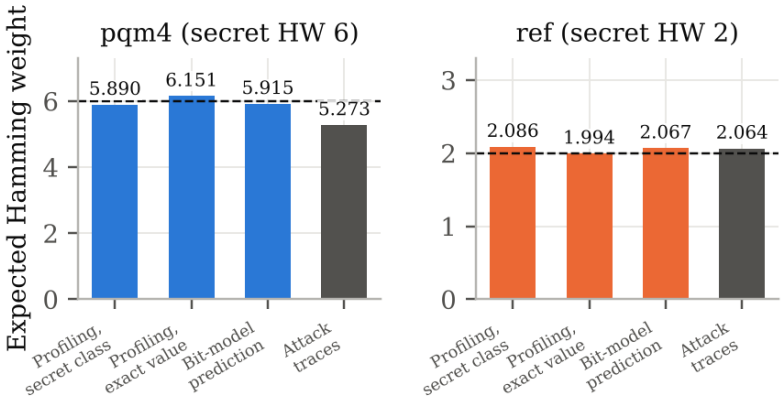


Fig. 7.5 Expected Hamming Weight by Trace Group, pqm4 and ref

The pqm4 failure is therefore a session-shift (domain-shift) problem and not a modelling one. A better model trained on the profiling set alone cannot fix it. It would take recordings from matching sessions or a method that adapts to the attack session, which would mean mixing attack data into training, and since the decision was to report the failure as found, no such adaptation was tried.

The bit-level model behind the third row regresses the expected HW on the 12 bits of the value. Its weights (Fig. 7.6) show how fully the model resolves each bit, where a weight of 1 means the bit counts exactly as it does in the Hamming weight.

The ref model recovers an almost perfect Hamming weight (all weights within 5% of 1, $R^2 = 0.984$). The pqm4 model sees each bit only partly (weights 0.41 to 0.82, $R^2 = 0.683$), with bit 7 the weakest, which fits its 39% accuracy: the pqm4 leakage at the chosen points is not a clean Hamming weight, so a 12-class HW label only captures part of it. A different leakage model for pqm4, such as per-bit labels or the value itself, is worth trying in future work.

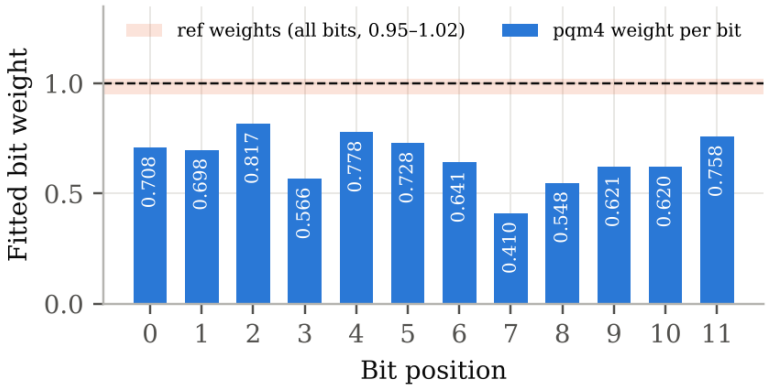


Fig. 7.6 Bit Weights of the Learned Leakage Model: pqm4 Bits and the ref Range

7.4 Findings on the EVT Cut-Off

The GPD cut-off shrank the point sets by 46% (pqm4) and 70% (ref) at a cost of 0.2 and 3.6 points of validation accuracy (Table 6.6) and, for ref, with no cost at all on the attack data (Table 7.3). This answers RQ2 positively for the unprotected implementations, with two qualifications from Chapter 6. The cut-off works only through the threshold u, because the score keeps the |t| ranking, and u depends on the tail fraction chosen (Table 6.3). The EVT cut-off is best seen as a data-driven stopping rule whose stopping point is a design parameter, not as a statistically calibrated detection threshold. Stage 11 then showed that the selection statistic and input scaling matter more than how many points are kept: with the class-weighted SNR and standardisation, 50 points are already more than enough.

7.5 Comparison With the Base Paper and Related Work

Table 7.5 places the results, beside the base paper and related work. Direct comparison is limited because the works differ in target, side channel, label and what counts as success. The dissertation recovers the HW class of one coefficient, not the key.

Table 7.5 Comparison With the Base Paper and Related Work

Work	Target and data	Label / goal	Reported result	Evaluated on independent attack data
Chinbat et al. (2025), base paper	FPGA Kyber NTT, 200k power traces	NTT intermediate classes	Up to 96.64% accuracy; 10.5× faster training with TVLA filtering	No (validation accuracy)
Ravi et al. (2023)	ARM Cortex-M4, Kyber pointwise multiplication	HW classifier, full key	820 traces with perfect HW classifier; about 7,805 at 95% accuracy	Yes
Magazin & Abdellatif (2026), dataset benchmark	Same dataset, pqm4 and mkm4	Full coefficient, CPA	pqm4 about 40 traces (textbook CPA); mkm4 about 200 traces (second-order CPA)	Yes
Feretzakis & Papaspyridis (2026)	Same dataset, pqm4 and ref	Leakage detection (binary)	Detection within 62–316 evaluation traces; $\|t\| > 4.5$ false alarms 2.7–12.9% under peeking	Detection task
This work, Stages 4–9	Same dataset, all three implementations	HW of coefficient 0, 12 classes	EVT cut-off: 46–70% fewer points, 0.2–3.6 points accuracy loss; ref HW class in 4 traces (corrected)	Yes, first 10,000 and full sets
This work, Stage 11	Same dataset, pqm4 and ref	HW of coefficient 0, 12 classes	ref: 94.77% validation, 94.65% attack, HW class from 1 trace (SR 0.892); pqm4: 39.61% validation, fails on attack set (session shift)	Yes, full sets

Next to the base paper the ref result achieves accuracy (94.77% compared to 96.64%) in a more difficult situation, software on a microcontroller with EM traces and includes an untouched test split and independent attack recordings where the accuracy remains strong. The base papers conclusion that point filtering does not

reduce accuracy is confirmed for ref and pqm4 on validation data (Stages 4 to 8) and, for ref on attack data.The pqm4 result shows why testing on attack data matters: a model 1.68 times better than baseline on profiling data picks the wrong class on every attack chunk, and validation accuracy alone would never show it.

7.6 Testing and Validation

Every stage was checked before its outputs were used. Table 7.6 lists each check, what it was meant to catch and how it turned out.

Table 7.6 Testing and Validation Checks

No.	Check	Purpose	Stage	Outcome
T1	Fixed-key constancy of mult_a / mult_b	Identify the secret operand	2	a constant (1 unique row in 200), b varies: a is secret
T2	ref byte decoding	Correct metadata encoding	2	Little-endian int16: 100% of values in [0, q); big-endian: 5.5%
T3	Masked share reconstruction	Correct masked secret	2, 10	b0 + b1 mod q equals the stored unmasked key
T4	Positive control on masked traces	Sensitivity of the t-test	3	Share-labelled |t| = 262.83 versus 3.20 for the secret
T5	Tail-fraction sensitivity	Stability of the GPD fit	5	ξ not stable; tail fraction reported as a design choice
T6	Null model	Prior-only baseline for GE	9	Revealed GE saturation for pqm4
T7	Row-pairing audit	Trace-label alignment	10	Masked misalignment found and fixed; pqm4 and ref passed
T8	Class proportions vs theoretical prior	Label correctness	11.0	Within 0.1 percentage point for all classes
T9	Majority and uniform baselines	Reference accuracy	11.0	23.61% / 23.37% and 8.33%
T10	Selection on training split only	No information leak into evaluation	11.2	Validation, test and attack data unused in selection

No.	Check	Purpose	Stage	Outcome
T11	Standardisation check	Correct scaling	11.4	Validation median offset below 0.01 standard deviations
T12	Drift check	Session shift	11.4, 11.4b	pqm4 attack offset 21× validation; ref 4×
T13	Three seeds	Seed variance	11.5	Standard deviations 0.05–0.46 points
T14	Untouched test split	Over-use of validation data	11.7	Test results consistent with validation
T15	Full attack sets	Cross-session performance	11.8	ref holds; pqm4 fails
T16	Expected-HW diagnosis and bit model	Cause of pqm4 failure	11.8c	Session shift, not value difficulty
T17	Inference batch fallback	Robust GPU inference	11.8	Batch 1,024, fallback 256 after cuDNN failure at 8,192

7.7 Limitations

- **One coefficient, one leakage model.** All labels are the HW of coefficient 0 of the secret, so the attack recovers one HW class rather than a coefficient or key, and the other 255 coefficients were not modelled. The pqm4 bit weights (Fig. 7.6) show that the HW model captures only part of the pqm4 leakage.
- **One secret per attack set.** Each fixed-key set has a single secret, so the attack-set results amount to one experiment per implementation, and the pqm4 diagnosis rests on this value and on the 12 profiling traces that share it.
- **Session shift not corrected.** The pqm4 failure was diagnosed but not fixed; whether domain adaptation or new recordings would rescue the attack is still open.

- **Unequal number of seeds.** Three seeds were used for the main configurations, but only one for the pqm4 CNN at K = 100/150 and for the ref MLP.
- **No ablation of Stage 11 changes.** The separate contributions of standardisation, the weighted SNR and model size to the Stage 11 gains were not measured.
- **EVT cut-off depends on a design parameter.** The cut-off depends on the tail fraction, and the GPD fit to |t| ignores the correlation between neighbouring samples (no declustering).
- **Masked implementation: first order only.** No second-order attack was tried, even though the dataset authors show that one succeeds in about 200 traces (Magazin & Abdellatif, 2026).
- **Some values not recorded.** A few summary values (marked — in Tables 7.2 and 7.3) were not saved during the runs and are left blank instead of being estimated.

7.8 Contributions

Measured against the literature in Chapter 2, the dissertation makes the following contributions, each stated together with what it does not claim.

- **A complete, audited assessment of all three implementations of a public ML-KEM EM dataset**, taking it from raw traces to labels, leakage statistics and attacks, and including the corrections the dataset needs (trace lengths, secret operand, ref byte encoding, masked label alignment). Feretzakis and Papaspyridis (2026) study the same pqm4 and ref recordings for detection only; this work adds profiled attacks and the masked implementation.
- **An empirical test of a GPD tail cut-off as a point-selection rule**, which cut the point sets by 46–70% at a small validation cost and no attack cost for ref. It keeps the |t| ranking and its stopping point depends on the tail fraction, so it is not a calibrated detection threshold.

- **A leakage-free maximum-accuracy profiled attack** (training-only selection, standardisation, three seeds, calibration, an untouched test split and full attack sets) that reaches 94.65% single-trace accuracy and recovers the HW class from one trace on the ref attack set, while showing that 50 points are enough.
- **A diagnosed cross-session failure.** The pqm4 attack is 1.68 times above baseline on profiling data yet fails on the attack set, and the expected-HW and bit-model diagnosis traces this to session shift rather than a weak model or a hard value. It is a practical warning against validation-only reporting of the kind used in the base paper.
- **A demonstration of prior effects on this dataset.** With plain scoring the old ref models pick the wrong class, while with prior correction they find the right one within 4 traces, and pqm4 guessing entropy saturates through the prior. This applies the analysis of Rousselot et al. (2026) to an ML-KEM dataset.

Taken together, these address gaps G1 to G7 of Table 2.3.

CHAPTER 8

SUMMARY

This dissertation studied the EM leakage of three ML-KEM implementations from the public Ledger Donjon dataset. It combined conventional leakage statistics, EVT and deep-learning profiled attacks, and tested every attack on independent fixed-key recordings.

Both unprotected implementations leak the HW of secret coefficient 0 strongly (maximum |t| of 168.4 for pqm4 and 55.2 for ref on the complete profiling sets). The first-order masked implementation shows no first-order leakage of the secret (maximum |t| = 3.20), even though each share leaks heavily (|t| = 262.8). A GPD tail cut-off on |t| reduced the points of interest by 46% and 70% at a validation cost of 0.2 and 3.6 percentage points and, for ref, with no cost on attack data.

A rebuilt, leakage-free attack protocol took ref from 50% to 94.8% validation accuracy, and that accuracy held on the full attack set (94.65%), where a single trace gives away the secret's HW class 89% of the time. Fifty points were enough for both implementations. The pqm4 attack, 1.68 times above baseline on profiling data, failed on the attack set, and the failure was traced to a shift between recording sessions rather than to the model or the secret value.

8.1 Objectives and Outcomes

Table 8.1 maps each objective of Section 1.1 to its outcome.

No.	Objective (Section 1.1)	Outcome	Status
1	HW-based leakage assessment on all three implementations	Complete profiling sets; strong leakage in pqm4 and ref; none of the secret in masked, with positive control	Achieved
2	GPD cut-off versus top-50 selection	46–70% fewer points; small validation cost; no ref attack cost; limits stated	Achieved
3	CNN profiled attacks on fixed-key traces	Stage 8 CNNs evaluated on first 10,000 and full attack sets	Achieved

No.	Objective (Section 1.1)	Outcome	Status
4	Ground-truth and pairing verification	Secret operand, encoding and masked alignment verified; one error found and fixed	Achieved
5	Maximum-accuracy attack and minimum points	ref 94.77% validation, 94.65% attack; K = 50 sufficient	Achieved
6	Full attack-set evaluation and failure diagnosis	ref succeeds from 1 trace; pqm4 failure diagnosed as session shift	Achieved (pqm4 attack not recovered)

Table 8.1 Objectives and Their Outcome

8.2 Answers to the Research Questions

- **RQ1.** pqm4 and ref leak strongly, peaking at samples 183 and 7917, while the masked implementation does not leak the secret at first order.
- **RQ2.** The GPD cut-off keeps 27 and 15 points instead of 50, at a cost of 0.2 and 3.6 points of validation accuracy, and its stopping point depends on the tail fraction.
- **RQ3.** The ref CNN reaches 94.1–94.8% and the pqm4 MLP 39.1–39.6% validation accuracy, with little change from K = 50 to 150; the ref result holds on the test split and the attack set, the pqm4 result on the test split only.
- **RQ4.** The pqm4 failure comes from a shift between the profiling and attack recording sessions.

8.3 Recommendations

The results lead to four recommendations for anyone evaluating deep-learning profiled attacks on ML-KEM.

- Report results on independent attack recordings, not just on validation data, and check for drift between profiling and attack data before training.

- When class priors are not uniform, as with HW labels, report a majority-class baseline and a null model, and score attacks with the prior removed.
- Choose points of interest on the training split only, using a statistic that separates all classes, and standardise inputs with training-split statistics.
- Calibrate model probabilities before using them for anything beyond the predicted class.

8.4 Future Work

- **C3: EVT applied to attack evidence.** Apply EVT to the per-trace evidence of the Stage 11 attacks, to describe how often single traces badly mislead an attack and how this differs between an attack that works and one that fails.
- **C1: calibrated detection thresholds.** Replace the fixed $|t| > 4.5$ rule and the quantile cut-off with a family-wise threshold taken from the distribution of the trace-wide maximum statistic under no leakage, allowing for correlation between neighbouring samples, building on Peschel et al. (2026), Murphy et al. (2023) and the designed nulls of Feretzakis and Papaspyridis (2026).
- **C2: tail-sensitive class tests.** Test whether class-conditional tail behaviour carries leakage that mean-based tests miss, taking into account the negative result of Meunier (2026) for masked AES.
- **pqm4 session shift.** Try domain adaptation or session-matched profiling, together with other leakage models (per-bit labels or values) suggested by Fig. 7.6.
- **Masked implementation.** Run a second-order analysis that combines both share windows and compare it with the bivariate CPA of the dataset authors.
- **C4: generalisation.** Extend the work from coefficient 0 to all 256 coefficients and to a second public dataset.

CHAPTER 9

REFERENCES

Baez, R., Olivas, A., Diamond, N. K., Frias, M., Noller, Y., & Tizpaz-Niari, S. (2025). *Risk estimation in differential fuzzing via extreme value theory* (arXiv:2511.02927). arXiv. https://doi.org/10.48550/arXiv.2511.02927

Béguinot, J., Rioul, O., Masure, L., Standaert, F.-X., Cheng, W., & Guilley, S. (2025). *Scalable information theoretic evaluation of the rank statistics in side-channel attacks* (Cryptology ePrint Archive, Paper 2025/1316). https://eprint.iacr.org/2025/1316

Chinbat, M., Wu, L., Zhang, X., Yang, Y., & Wei, M. (2025). Comparative deep learning-based side-channel analysis of an FPGA-based CRYSTALS-Kyber NTT accelerator. *Cryptography, 9*(4), Article 64. https://doi.org/10.3390/cryptography9040064

des Noes, M. (2024). Distribution of signal to noise ratio and application to leakage detection. *IACR Transactions on Cryptographic Hardware and Embedded Systems, 2024*(2), 384–402. https://doi.org/10.46586/tches.v2024.i2.384-402

Dubrova, E., Ngo, K., Gärtner, J., & Wang, R. (2023). Breaking a fifth-order masked implementation of CRYSTALS-Kyber by copy-paste. In *Proceedings of the 10th ACM Asia Public-Key Cryptography Workshop (APKC '23)* (pp. 10–20). ACM. https://doi.org/10.1145/3591866.3593072

Feretzakis, G., & Papaspyridis, A. (2026). *The price of peeking: Anytime-valid leakage detection on ML-KEM EM traces* (arXiv:2609.33597). arXiv. https://doi.org/10.48550/arXiv.2609.33597

Ganesh, B., Ahmed, M. M., & Mady, A. (2024). Higher order leakage assessment and neural network-based attack on CRYSTALS-Kyber. In *Proceedings of the 21st International Conference on Security and Cryptography (SECRYPT 2024)* (pp. 373–380). SciTePress. https://doi.org/10.5220/0012715700003767

Kennaway, M., Hoang, T., Khalid, A., Rafferty, C., & O'Neill, M. (2025). An enhanced two-step CPA side-channel analysis attack on ML-KEM. In *Proceedings of the 22nd International Conference on Security and*

Cryptography (SECRYPT 2025) (pp. 263–274). SciTePress. https://doi.org/10.5220/0013638600003979

Magazin, A. A., & Abdellatif, K. M. (2026). *Open EM side-channel dataset for ML-KEM (Kyber) implementations* (Cryptology ePrint Archive, Paper 2026/1851). https://eprint.iacr.org/2026/1851

Meunier, Q. L. (2026). *A unified framework for statistical side-channel distinguishers: From leakage assessment to key recovery* (Cryptology ePrint Archive, Paper 2026/2041). https://eprint.iacr.org/2026/2041

Mikulec, J., Breier, J., & Hou, X. (2026). *Beyond TVLA: Anderson–Darling leakage assessment for neural network side-channel leakage detection* (arXiv:2603.18647). arXiv. https://doi.org/10.48550/arXiv.2603.18647

Murphy, C., Tawn, J. A., & Varty, Z. (2023). *Automated threshold selection and associated inference uncertainty for univariate extremes* (arXiv:2310.17999). arXiv. https://doi.org/10.48550/arXiv.2310.17999

National Institute of Standards and Technology. (2024). *Module-lattice-based key-encapsulation mechanism standard* (Federal Information Processing Standards Publication 203). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.203

Nouraniboosjin, S., & Ganji, F. (2024). *Too hot to be true: Temperature calibration for higher confidence in NN-assisted side-channel analysis* (Cryptology ePrint Archive, Paper 2024/071). https://eprint.iacr.org/2024/071

Nouraniboosjin, S., & Ganji, F. (2026). *How many traces suffice? PAC guarantees for profiled and non-profiled side-channel analysis* (Cryptology ePrint Archive, Paper 2026/1770). https://eprint.iacr.org/2026/1770

Peschel, S., Boulesteix, A.-L., von Mutius, E., & Müller, C. L. (2026). *permApprox: A general framework for accurate permutation p-value approximation* (arXiv:2602.22975). arXiv. https://doi.org/10.48550/arXiv.2602.22975

Picek, S., Perin, G., Mariot, L., Wu, L., & Batina, L. (2023). SoK: Deep learning-based physical side-channel analysis. *ACM Computing Surveys, 55*(11), 1–35. https://doi.org/10.1145/3569577

Ravi, P., Jap, D., Bhasin, S., & Chattopadhyay, A. (2023). Invited paper: Machine learning based blind side-channel attacks on PQC-based KEMs – A case study of Kyber KEM. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)* (pp. 1–7). IEEE. https://doi.org/10.1109/ICCAD57390.2023.10323721

Rousselot, N., Heydemann, K., Masure, L., Migairou, V., & Strullu, R. (2026). *Is it really broken? The failure of DL-SCA scoring metrics under non-uniform priors* (Cryptology ePrint Archive, Paper 2026/338). https://eprint.iacr.org/2026/338

Anthropic. (2026). *Claude* [Large language model]. https://claude.ai

APPENDIX A

This appendix contains sample code (A.1), additional tables (A.2), and a map of the notebook stages and saved outputs with a glossary (A.3). In line with the report format, only sample code is given, not the complete notebook. The excerpts are simplified for reading: progress printing, path handling and error handling are removed.

A.1 Sample Code

A.1.1 Point-of-Interest Selection With Minimum-Distance Exclusion (Stages 4 and 6)

```
def select_poi(score_array, k=50, min_distance=20, use_abs=True,
stop_at_zero=False):
    scores = np.abs(score_array) if use_abs else score_array.copy()
    scores = scores.copy()
    selected = []
    for _ in range(k):
        idx = np.argmax(scores)
        if scores[idx] <= -np.inf:
            break
        if stop_at_zero and scores[idx] <= 0:
            break
        selected.append(idx)
        lo = max(0, idx - min_distance)
        hi = min(len(scores), idx + min_distance + 1)
        scores[lo:hi] = -np.inf
    return np.array(selected)
```

A.1.2 Hamming-Weight Labels and Metadata Decoding (Stages 2 and 11.0)

```
Q = 3329

def hw(x):
    x = np.asarray(x, dtype=np.uint16)
    return np.unpackbits(x.view(np.uint8).reshape(-1, 2), axis=1).sum(1)

def load_secret_coeff0(meta_file, byte_encoded=False):
    a = np.load(meta_file)
    if byte_encoded:                # ref: 512 uint8 -> 256 little-endian int16
        a = a.view('<i2')
    c0 = a[:, 0].astype(np.int64)
```

```
    assert ((c0 >= 0) & (c0 < Q)).all()
    return c0

# majority-class baseline (Stage 11.0)
counts = np.bincount(labels, minlength=12)
baseline = counts.max() / counts.sum()   # 0.2361 pqm4, 0.2337 ref
```

A.1.3 Stratified Split (Stage 11.1)

```
from sklearn.model_selection import train_test_split

idx = np.arange(len(labels))
tr, tmp = train_test_split(idx, test_size=0.2, stratify=labels, random_state=42)
va, te = train_test_split(tmp, test_size=0.5, stratify=labels[tmp], random_state=42)
np.savez(split_file, train=tr, val=va, test=te)
```

A.1.4 Standardisation and Drift Check (Stage 11.4)

```
mu = X_train.mean(axis=0)
sd = X_train.std(axis=0) + 1e-8

def standardise(X):
    return (X.astype(np.float32) - mu) / sd

def drift(X):
    z = standardise(X)
    return np.median(np.abs(z.mean(axis=0)))

print('val', drift(X_val), 'attack', drift(X_attack))
# pqm4: 0.0058 vs 0.120   ref: 0.0078 vs 0.0288
```

A.1.5 Training Loop With Callbacks and Resumable Saving (Stage 11.5)

```
def train_one(build_fn, k, seed, Xtr, ytr, Xva, yva, out_path):
    if os.path.exists(out_path):          # resume after disconnection
        return tf.keras.models.load_model(out_path)
    tf.keras.utils.set_random_seed(seed)
    model = build_fn(k)
    model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
                  loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
                  metrics=['accuracy'])
    cb = [tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=10,
                                           restore_best_weights=True),
          tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5,
patience=4)]
    model.fit(Xtr, ytr, validation_data=(Xva, yva), epochs=100,
              batch_size=512, callbacks=cb, verbose=2)
```

```
    model.save(out_path)
    return model
```

A.1.6 Temperature Scaling (Stage 11.6)

```
from scipy.optimize import minimize_scalar
from scipy.special import log_softmax

def fit_temperature(logits_val, y_val):
    def nll(T):
        lp = log_softmax(logits_val / T, axis=1)
        return -lp[np.arange(len(y_val)), y_val].mean()
    return minimize_scalar(nll, bounds=(0.05, 20.0), method='bounded').x
```

A.1.7 Batched Prediction With Fallback (Stages 11.7–11.8)

```
def predict_logits(model, X, batch=1024):
    try:
        return model.predict(X, batch_size=batch, verbose=0)
    except Exception:                    # cuDNN autotune failure on large batches
        return model.predict(X, batch_size=256, verbose=0)
```

A.1.8 Confusion Matrix (Stage 11.7)

```
def confusion(y_true, y_pred, n=12):
    cm = np.zeros((n, n))
    np.add.at(cm, (y_true, y_pred), 1)
    return cm / np.maximum(cm.sum(1, keepdims=True), 1)   # row-normalised
```

A.1.9 Guessing Entropy With a Null Model (Stage 9)

```
def guessing_entropy(log_probs, true_class, n_repeats=20, max_traces=200):
    n = log_probs.shape[0]; max_traces = min(max_traces, n)
    ranks = np.zeros((n_repeats, max_traces))
    for r in range(n_repeats):
        order = np.random.permutation(n)[:max_traces]
        cum = np.cumsum(log_probs[order], axis=0)
        for t in range(max_traces):
            ranks[r, t] = np.where(np.argsort(-cum[t]) == true_class)[0][0]
    return ranks.mean(axis=0)

# null model: every trace gets the training class frequencies
null_log_probs = np.tile(np.log(train_class_freq), (n_attack, 1))
```

A.1.10 Streaming SNR and t-Test Accumulators (Stage 3)

```
def stream_snr_t(trace_files, label_files, n_classes=12, chunk=10_000):
    S = S2 = None; N = np.zeros(n_classes)
    for tf, lf in zip(trace_files, label_files):
```

```
    X = np.load(tf, mmap_mode='r'); y = np.load(lf)
    for s in range(0, len(y), chunk):
        xb = np.asarray(X[s:s+chunk], dtype=np.float64); yb = y[s:s+chunk]
        if S is None:
            S = np.zeros((n_classes, xb.shape[1])); S2 = np.zeros_like(S)
        for c in range(n_classes):
            m = yb == c
            N[c] += m.sum(); S[c] += xb[m].sum(0); S2[c] += (xb[m]**2).sum(0)
  mu = S / N[:, None]; var = S2 / N[:, None] - mu**2
  snr = mu.var(axis=0) / var.mean(axis=0)
  return snr, mu, var, N
```

A.1.11 MLP Construction (Stage 11.5)

```
def build_mlp(k, n_classes=12, l2=1e-4):
  reg = tf.keras.regularizers.l2(l2)
  inp = tf.keras.Input(shape=(k, 21))       # K windows x 21 samples
  x = tf.keras.layers.Flatten()(inp)
  for units in (256, 128):
      x = tf.keras.layers.Dense(units, kernel_regularizer=reg)(x)
      x = tf.keras.layers.BatchNormalization()(x)
      x = tf.keras.layers.ReLU()(x)
      x = tf.keras.layers.Dropout(0.3)(x)
  x = tf.keras.layers.Dense(64, activation='relu', kernel_regularizer=reg)(x)
  out = tf.keras.layers.Dense(n_classes)(x)   # logits
  return tf.keras.Model(inp, out)
```

A.2 Additional Tables

Table A.1 lists the Hamming-weight class of each fixed secret and its prior probability, which determines how easily a prior-biased model can appear to succeed.

Table A.1 Fixed Secrets and Their Prior Probability

Implementation	Fixed secret coefficient 0	Binary (12 bits)	HW class	Prior probability of class	Rank of class by prior
pqm4	679	0010 1010 0111	6	0.2355	1 (most common)
ref	1026	0100 0000 0010	2	0.0198	8
masked	1681	0110 1001 0001	5	0.2187	2

Table A.2 compares the two generations of attack models side by side.

Table A.2 Stage 8 and Stage 11 Attack Configurations

Setting	Stage 8 (Review 2)	Stage 11 (final)
Point ranking statistic	Welch |t|, HW ≤ 6 vs > 6, full profiling set	Class-weighted SNR, training split only
Number of points	50 (conventional) or 27 / 15 (EVT)	50, 100, 150 (nested prefixes of one ranked list)
Input	Raw int16	Standardised with training statistics
Split	90/10 train/validation	80/10/10 train/validation/test
Models	Small CNN, 4,716 parameters	MLP (0.31–0.85 M) and CNN (0.50–1.32 M parameters)
Output	Softmax	Logits + temperature scaling
Batch, epochs	256, up to 50	512, up to 100
Early stopping	Validation accuracy, patience 10	Validation loss, patience 10, with LR reduction
Seeds	1	3 (main configurations)
Attack data	First 10,000 traces (Stage 9), full set (Stage 11.9)	Full set
Scoring	Plain and null-corrected	Plain and prior-corrected

Table A.3 restates the meaning of the shape parameter, used in Section 6.1.4.

Table A.3 Interpreting the GPD Shape Parameter

Shape ξ	Tail type	Meaning for the |t| tail (Stage 5)
$\xi > 0$	Heavy (power-law)	Large |t| values are relatively common among the strongest samples and the tail decays slowly (pqm4, $\xi = 0.564$)
$\xi = 0$	Exponential	The tail decays exponentially; ref is close to this case ($\xi = 0.081$)
$\xi < 0$	Bounded	The tail has a finite upper end; no sample can exceed

A.3 Notebook Map, Saved Outputs and Glossary

All work was done in a Colab notebook (capstone.ipynb). The notebook is organized by stage. Table A.4 shows each stage in relation to the notebook section.

What it stores in Drive so that every reported number can be traced back, to its source.

Table A.4 Notebook Stages and Saved Outputs

Stage	Notebook section	Saved outputs	Used by
1–2	Setup, inventory, ground truth	Verified secrets, decoded operands	All stages
3	Full-dataset streaming SNR / t-test (cell 60 at Review 2)	Per-sample |t| and SNR arrays per implementation	4–6
3 (masked)	Masked secret vs share control (cell 30 at Review 2)	Masked |t| arrays	10
4	Conventional POI selection	baseline_pois.npz	8–9
5–6	GPD fit and EVT POI selection (cells 62–63 at Review 2)	GPD parameters, EVT POI sets	8–9
7–8	CNN design and training (cell 65 at Review 2)	Trained Stage 8 models	9, 11.9
9	Attack evaluation, first 10,000 traces	GE curves, advantage table	Chapter 6
10	Integrity audit	Rebuilt masked labels	3–9 (masked)
11.0–11.1	Data checks and split	Class counts, split indices	11.2–11.8
11.2–11.3	Weighted SNR and features	Ranked top-150 points, feature arrays	11.4–11.8
11.4–11.4b	Standardisation and drift	Training mean/std, drift summary	11.5–11.8
11.5–11.6	Training and calibration	Models per K and seed, histories, temperatures	11.7–11.9
11.7	Test evaluation	Confusion matrices	Chapter 7
11.8–11.8c	Attack evaluation and diagnosis	Attack log-probabilities, SR and GE, expected-HW tables	Chapter 7
11.9–11.10	Old vs new comparison	Comparison table and figure	Chapter 7

Table A.5 gives the directory layout of the extracted dataset on Drive (Section 3.3); {i} indexes the numbered chunks.

Table A.5 Dataset Directory Structure

Path (under d0nj0n_mlkem_dataset/)	Contents
pqm4-ref/pqm4/ fixed_key/leakages/	traces_{i}.npy: pqm4 attack traces, int16, 10,000 × 20,000 per chunk
pqm4-ref/pqm4/ fixed_key/metadata/	mult_a_{i}.npy, mult_b_{i}.npy, mult_{i}.npy: operands and product, int16, 10,000 × 256
pqm4-ref/ref/fixed_key/ leakages/	traces_{i}.npy: ref attack traces, 10,000 × 13,000 per chunk
pqm4-ref/ref/fixed_key/ metadata/	mult_a_{i}.npy, mult_b_{i}.npy, mult_{i}.npy: raw bytes (uint8, 10,000 × 512), read as int16
pqm4-ref/pqm4/, pqm4-ref/ref/ (variable-key directories)	Profiling traces and metadata in the same layout
masked/ 100k_capture_all_2/	Masked profiling set: traces/traces_s0_{i}.npy, traces_s1_{i}.npy, trigs_s0_{i}.npy, trigs_s1_{i}.npy; metadata/ap_{i}.npy, bp_s0_{i}.npy, bp_s1_{i}.npy, ct_{i}.npy, masked_keys_{i}.npy
masked/ 100k_capture_fixedkey_all/	Masked attack set: same layout plus metadata/ fixed_unmasked_key.npy

Glossary

- **Attack set (fixed-key set):** traces recorded with one constant secret, used only for evaluation.
- **Base multiplication (basemul):** the degree-one polynomial products that make up NTT-domain multiplication in ML-KEM.
- **Calibration:** adjusting predicted probabilities so that they match observed frequencies; here by temperature scaling.

- **Drift (session shift):** a difference between the distribution of profiling and attack recordings that is not caused by the secret.
- **Generalised Pareto Distribution (GPD):** the limiting distribution of excesses over a high threshold (Eqn. 1.3).
- **Guessing entropy (GE):** average rank of the true class after n traces; 0 means ranked first.
- **Hamming weight (HW):** number of set bits in a value.
- **Null model:** a model that outputs the training class frequencies for every trace, ignoring the trace.
- **Point of interest (POI):** a sample index selected as input to an attack.
- **Prior correction:** removing log $\pi(c)$ from each class score (Eqn. 1.6).
- **Profiling set (variable-key set):** traces with a fresh known secret per trace, used for training.
- **Shape parameter ξ:** the GPD parameter that sets the tail type (Table A.3).
- **Success rate (SR):** fraction of independent attacks with n traces whose top-ranked class is correct.
- **Weighted SNR:** SNR in which classes are weighted by their frequency (Eqn. 1.2).